



## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams  
File Open / Close

Formatted I/O  
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

# Programming in Modern C++

## Module M41: Input-Output: File Handling in C

Partha Pratim Das

Department of Computer Science and Engineering  
Indian Institute of Technology, Kharagpur

*ppd@cse.iitkgp.ac.in*

*All url's in this module have been accessed in September, 2021 and found to be functional*

Many diagrams in this module are taken from *Computer Science: A Structured Programming Approach Using C*



# Weekly Recap

## Module M41

Partha Pratim Das

### Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

- Introduced the concept of exceptions
  - Discussed error handling in C with various language features and library support in C for handling errors
  - Discussed exception (error) handling in C++ with `try-throw-catch` feature in C++ for handling errors
- Introduced the templates in C++
  - Discussed function templates as generic algorithmic solution for code reuse
  - Discussed class templates as generic solution for data structure reuse
  - Explained partial template instantiation and default template parameters
  - Demonstrated templates on inheritance hierarchy
- Introduced Function Objects or Functors
  - Illustrated functors with several simple examples and examples from STL



# Module Objectives

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

- Understand file handling and I/O in C
- To understand Text and Binary I/O



# Module Outline

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams  
File Open / Close

Formatted I/O  
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

- 1 Weekly Recap
- 2 Standard Library for I/O
- 3 Files and Streams
  - File Open / Close
- 4 Formatted I/O
  - Output
  - Read
- 5 Unformatted I/O
- 6 Direct IO
- 7 File Positioning
- 8 Module Summary



# Standard Library for I/O

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

## Standard Library for I/O

### Sources:

- [Computer Science: A Structured Programming Approach Using C](#)
- [C file input/output, Wikipedia](#)



# Standard C I/O Functions

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams  
File Open / Close

Formatted I/O  
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

- The C programming language provides many standard library functions for file input and output. These functions make up the bulk of the C standard library header `<stdio.h>`
- **Categories of I/O Functions**
  - File Open/Close
  - Formatted Input/Output
  - Character Input/Output
  - Line Input/Output
  - Block Input/Output
  - File Positioning
  - System File Operations
  - File Status

**Source:** [C file input/output](#), Wikipedia



# Files and Streams

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

**Files and Streams**

File Open / Close

Formatted I/O

Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

# Files and Streams



# Files and Streams

Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

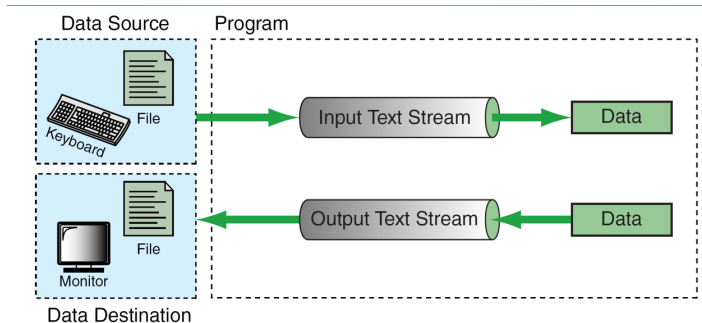
Unformatted I/O

Direct IO

File Positioning

Module Summary

- *A **file** is an external collection of related data treated as a unit.* The primary purpose of a file is to keep a record of data. Since the contents of primary memory are lost when the computer is shut down, we need files to store our data in a more permanent form
- *Data is input to and output from a **stream**.* A stream can be associated with a physical device, such as a terminal, or with a file stored in auxiliary memory







# Reading and Writing Text Files

## Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

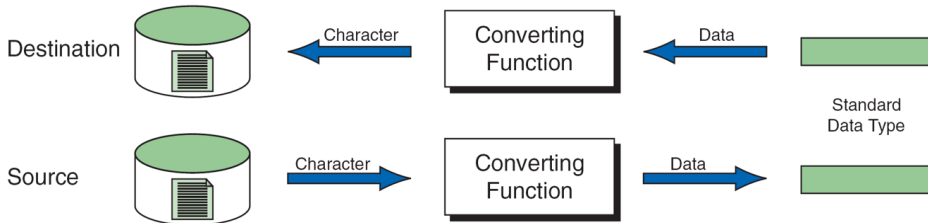
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary



Text files are used for:

- Formatted input/output functions
- Character input/output functions
- String input/output functions



# Binary File: Block Input and Output

## Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

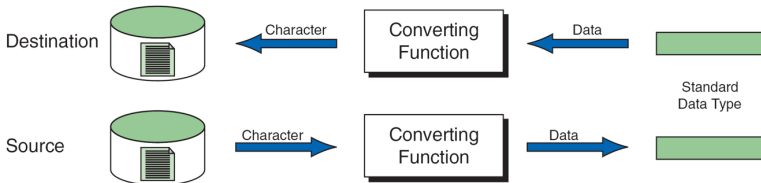
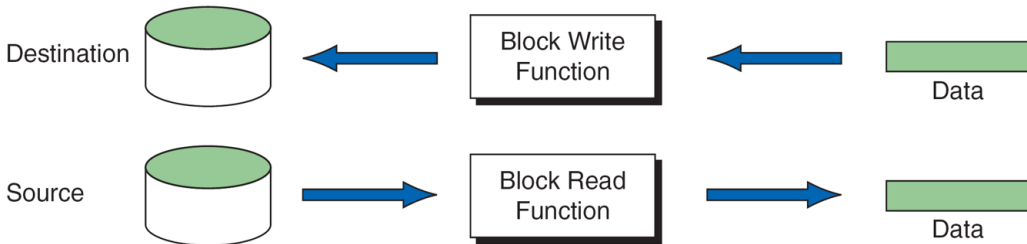
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary





# Text and Binary Files

Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

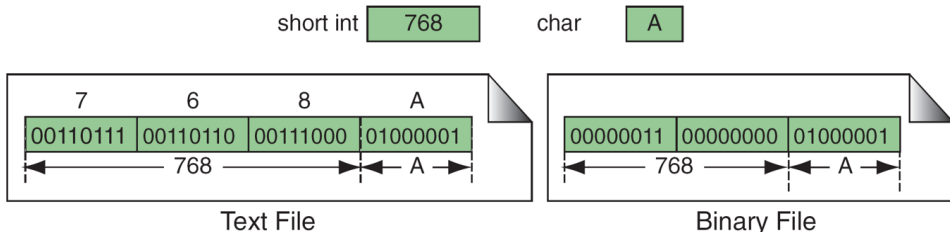
Unformatted I/O

Direct IO

File Positioning

Module Summary

- `ascii(7) = 55 = 0b 0011 0111`
- `ascii(6) = 54 = 0b 0011 0110`
- `ascii(8) = 56 = 0b 0011 1000`
- `ascii('A') = 65 = 0b 0100 0001`
- `768 = 0b 0000 0011 0000 0000`



- Text files store data as a sequence of characters
- Binary files store data as they are stored in primary memory



# File Modes

Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

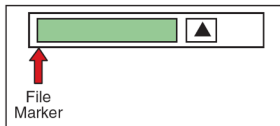
File Positioning

Module Summary

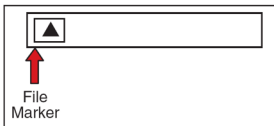
| Mode                                  | r    | w     | a     | r+   | w+    | a+    |
|---------------------------------------|------|-------|-------|------|-------|-------|
| <b>Open state</b>                     | read | write | write | read | write | write |
| <b>Read allowed</b>                   | yes  | no    | no    | yes  | yes   | yes   |
| <b>Write allowed</b>                  | no   | yes   | yes   | yes  | yes   | yes   |
| <b>Append allowed</b>                 | no   | no    | yes   | no   | no    | yes   |
| <b>File must exist</b>                | yes  | no    | no    | yes  | no    | no    |
| <b>Contents of existing file lost</b> | no   | yes   | no    | no   | yes   | no    |

For read/write of binary files, use 'b' with one of the above modes

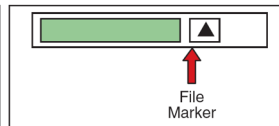
## File Opening Modes



Read Mode (r, r+)



Write Mode (w, w+)



Append Mode (a, a+)



# File States and Transitions

## Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

## Files and Streams

File Open / Close

Formatted I/O

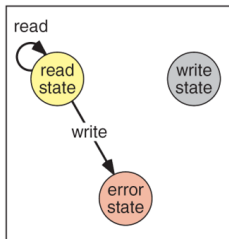
Output  
Read

Unformatted I/O

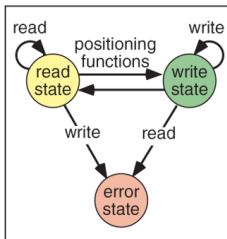
Direct IO

File Positioning

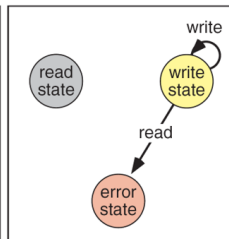
Module Summary



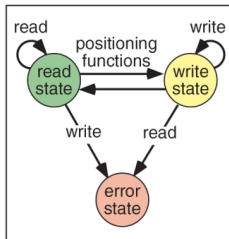
read mode (r)



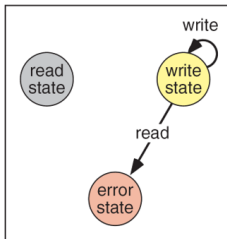
read update mode (r+)



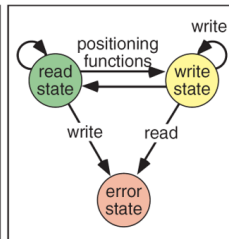
write mode (w)



write update mode (w+)



append mode (a)



append update mode (a+)



# File Open and Close

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams  
File Open / Close

Formatted I/O  
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

- To write to or read from a file, we need to **open** it:

```
FILE *fopen(const char *filename, const char *mode);
```

```
// Each FILE object denotes a C stream and keeps the state during I/O  
// filename: file name to associate the file stream to  
// mode: null-terminated character string determining file access mode
```

- If successful, returns a pointer to the new file stream. The stream is fully buffered unless filename refers to an interactive device.
- On error, returns a null pointer
- On successful opening, we **write** and/or **read** data using I/O functions
- Once the write or read file over, we need to **close** it:

```
int fclose(FILE *stream); // stream: the file stream to close
```

- Returns 0 on success, EOF otherwise // EOF is special End-of-File marker
- Closes the file stream, flushes unwritten buffered data, and discards unread buffered data
- The stream is no longer associated with a file, the buffer is disassociated and deallocated
- The behavior is undefined if the value of the pointer stream is used after fclose returns



# File Open and Close

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    // ...
    FILE* spTemps; // Declarations for file handler

    // ...

    if ((spTemps = fopen("TEMPS.DAT", "r")) == NULL) {
        printf("\aERROR opening TEMPS.DAT\n");
        exit(100);
    } // if open

    // Perform I/O

    if (fclose(spTemps) == EOF) {
        printf("\aERROR closing TEMPS.DAT\n");
        exit(102);
    } // if close

    // ...
} // main
```



# Formatted I/O

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams  
File Open / Close

**Formatted I/O**  
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

## Formatted I/O





# Formatted I/O: File Write and Read

## Module M41

Partha Pratim Das

## Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

- To **write**, we use:

```
int printf(const char *format, ... );           // Writes to output stream stdout
int fprintf(FILE *stream, const char *format, ... ); // Writes to output stream stream
int sprintf(char *buffer, const char *format, ... ); // Writes to a string buffer
```

```
// format: A null-terminated multibyte string specifying interpretation of the data
// ...: arguments specifying data to print - a variadic function
// stream must be open before writing (stdout stays open) or reading (stdin stays open)
// buffer must be allocated before writing
```

- If successful, number of characters transmitted to the output stream or number of characters written to buffer (not counting the terminating null character) is returned
- A negative value is returned for an output error or an encoding error

- To **read**, we use:

```
int scanf(const char *format, ... );           // Reads from input stream stdin
int fscanf(FILE *stream, const char *format, ... ); // Reads from input stream stream
int sscanf(char *buffer, const char *format, ... ); // Reads from a string buffer
```

- Number of receiving arguments successfully assigned (which may be zero in case a matching failure occurred before the first receiving argument was assigned), or **EOF** if input failure occurs before the first receiving argument was assigned



# Side Effect and Value of Formatted I/O Function

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams  
File Open / Close

Formatted I/O  
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

- `printf`, `fprintf` etc.
  - **Side Effect**
    - ▷ Converts internal data, as required, to strings of characters and writes the converted values to a file, which may be the standard output or error file
  - **Value**
    - ▷ Returns the number of characters written to the output file. In case of an error, it returns EOF
- `scanf`, `fscanf` etc.
  - **Side Effect**
    - ▷ Reads and converts a stream of characters from the input file, and stores the converted values in the list of variables found in the address list
  - **Value**
    - ▷ Returns the number of successful data conversions. If end of file is reached before any data are converted, it returns EOF



# Format (Conversion) Specifications

Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

## scanf/fscanf



\* Suppress

h short  
l long int  
l double  
L long double

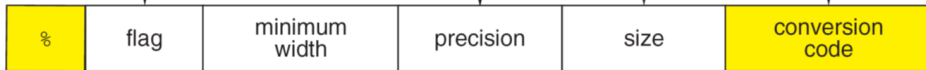
d, i, u, o, x,  
c, s, p, n,  
a, f, e, g, [

- left justify  
+ sign ( + or - )  
space if positive  
0 zero padding

.m

hh char  
h short  
l long int  
L long double

d, i, u, o, x, X,  
f, e, E, g, G, a, A,  
c, s, p, n, %



## printf/fprintf



# Print Built-in Type Data

## Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

```
#include <stdio.h>

int main() {
    int i = 17;
    long l = 0x012a78cb; // 19560651
    long long unsigned int i64 = 0x012a78cb2597ac3d; // 84012356964166717
    float f = 15.0 / 7;
    double d = 15.0 / 7;
    char c = 'x';
    const char *s = "ppd";
    int *p = &i;

    printf("%d\n", i); // dec // 17
    printf("%x\n", i); // hex // 11
    printf("%o\n", i); // oct // 21
    printf("%ld\n", l); // long // 19560651
    printf("%llu\n", i64); // int 64 // 84012356964166717
    printf("%f\n", f); // float // 2.142857
    printf("%lf\n", d); // double // 2.142857
    printf("%c\n", c); // char // x
    printf("%s\n", s); // string // ppd
    printf("%p\n", p); // pointer // 0x7ffc28102988
}
```



# Print User-defined Type Data

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

```
#include <stdio.h>
```

```
typedef struct Complex {  
    double re, im;  
} Complex;
```

```
int main() {  
    Complex c1 = { 2.5, 7.3 }, c2 = { 4.3, 8.9 };
```

```
    printf("(%lf, %lf)", c1.re, c1.im);    // Need to print component-wise
```

```
    printf("; ");
```

```
    printf("(%lf, %lf)\n", c2.re, c2.im);    // Need to print component-wise
```

```
    printf("(%lf, %lf)", c1);    // warning: format '%lf' expects argument of type 'double',  
                                // but argument 2 has type 'Complex' {aka 'struct Complex'}  
                                // warning: format '%lf' expects a matching 'double' argument
```

```
    printf("; ");
```

```
    printf("(%lf, %lf)\n", c2); // Same as above
```

```
}
```

```
(2.500000, 7.300000); (4.300000, 8.900000)
```

```
(2.500000, 7.300000); (4.300000, 8.900000)
```

- MSVC++ does not produce the warnings



# Flags, Sizes, and Conversion Code for printf family

## Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

| Argument Type       | Flag              | Size Specifier                                             | Code |
|---------------------|-------------------|------------------------------------------------------------|------|
| integer             | -, +, 0, space    | hh (char), h (short), none (int), l (long), ll (long long) | d, i |
| unsigned int        | -, +, 0, space    | hh (char), h (short), none (int), l (long), ll (long long) | u    |
| integer (octal)     | -, +, 0, #, space | hh (char), h (short), none (int), l (long), ll (long long) | o    |
| integer (hex)       | -, +, 0, #, space | hh (char), h (short), none (int), l (long), ll (long long) | x, X |
| real                | -, +, 0, #, space | none (double), l (double), L (double)                      | f    |
| real (scientific)   | -, +, 0, #, space | none (double), l (double), L (double)                      | e, E |
| real (scientific)   | -, +, 0, #, space | none (double), l (double), L (double)                      | g, G |
| real (hex)          | -, +, 0, #, space | none (double), l (double), L (double)                      | a, A |
| character           | -                 | none (char), l (wchar_t)                                   | c    |
| string              | -                 | none (char string), l (wchar_t string)                     | s    |
| pointer             |                   |                                                            | p    |
| integer (for count) |                   | none (int), h (short), l (long)                            | n    |
| to print %          |                   |                                                            | %    |



# Flag Formatting Options

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams  
File Open / Close

Formatted I/O  
**Output**  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

| Flag Type     | Flag Code | Formatting                                                            |
|---------------|-----------|-----------------------------------------------------------------------|
| Justification | none      | right justified                                                       |
|               | -         | left justified                                                        |
| Padding       | none      | space padding                                                         |
|               | 0         | zero padding                                                          |
| Sign          | none      | positive value: no sign<br>negative value: -                          |
|               | +         | positive value: +<br>negative value: -                                |
|               | space     | positive value: space<br>negative value: -                            |
| Alternate     | #         | print alternative format for<br>scientific, hexadecimal, and<br>octal |



# Read Built-in Type Data

## Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

```

#include <stdio.h>
int main() {
    int i; long l;
    long long unsigned int i64; // For a 64-bit machine
    float f; double d;
    char c; char *s = (char*)malloc(10); // Space needs to be allocated to store the string to be read
    int *p;

    // Input shown in magenta and Output shown in gray

    scanf("%d\n", &i);      printf("%d\n", i);      // dec      // 17 17
    scanf("%x\n", &i);      printf("%x\n", i);      // hex       // 11 11
    scanf("%o\n", &i);      printf("%o\n", i);      // oct       // 21 21
    scanf("%ld\n", &l);      printf("%ld\n", l);      // long      // 19560651 19560651
    scanf("%llu\n", &i64);    printf("%llu\n", i64);    // int 64    // 84012356964166717 84012356964166717
    scanf("%f\n", &f);       printf("%f\n", f);       // float     // 2.142857 2.142857
    scanf("%lf\n", &d);       printf("%lf\n", d);       // double    // 2.142857 2.142857
    scanf("%c\n", &c);       printf("%c\n", c);       // char      // x x

    // Used just 's', not &s, as it is a pointer
    scanf("%s\n", s);        printf("%s\n", s);        // string     // ppd ppd

    scanf("%p\n", &p);       printf("%p\n", p);       // pointer    // 008FFC0C 008FFC0C
}

```





# Sizes and Conversion Code for scanf family

## Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

| Argument Type       | Size Specifier                                                                                                                                              | Code     |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| integral            | <b>hh</b> (char), <b>h</b> (short), none (int), <b>l</b> (long), <b>ll</b> (long long) <b>h</b> (short), none (int), <b>l</b> (long). <b>ll</b> (long long) | <b>i</b> |
| integer             | <b>h</b> (short), none (int), <b>l</b> (long), <b>ll</b> (long long)                                                                                        | <b>d</b> |
| unsigned int        | <b>hh</b> (char), <b>h</b> (short), none (int), <b>l</b> (long), <b>ll</b> (long long)                                                                      | <b>u</b> |
| character octal     | <b>hh</b> (unsigned char)                                                                                                                                   | <b>o</b> |
| integer hexadecimal | <b>h</b> (short), none (int), <b>l</b> (long), <b>ll</b> (long long)                                                                                        | <b>x</b> |
| real                | none (double), <b>l</b> (double), <b>L</b> (double)                                                                                                         | <b>f</b> |
| real (scientific)   | none (double), <b>l</b> (double), <b>L</b> (double)                                                                                                         | <b>e</b> |
| real (scientific)   | none (double), <b>l</b> (double), <b>L</b> (double)                                                                                                         | <b>g</b> |
| real (hexadecimal)  | none (double), <b>l</b> (double), <b>L</b> (double)                                                                                                         | <b>a</b> |
| character           | none (char), <b>l</b> (wchar_t)                                                                                                                             | <b>c</b> |
| string              | none (char string), <b>l</b> (wchar_t string)                                                                                                               | <b>s</b> |
| pointer             |                                                                                                                                                             | <b>p</b> |
| integer (for count) | none (int), <b>hh</b> (char), <b>h</b> (short), <b>l</b> (long), <b>ll</b> (long long)                                                                      | <b>n</b> |
| set                 | none (char), <b>l</b> (wchar_t)                                                                                                                             | <b>[</b> |



# Checking scanf Results

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

```
#include <stdio.h>

#define FLUSH while (getchar() != '\n')
#define ERR1 "\aPrice incorrect. Re-enter both fields\n"
#define ERR2 "\aAmount incorrect. Re-enter both fields\n"

int main() {
    int amount;
    double price;
    int ioResult;

    // Read price and amount
    do {
        printf("\nEnter amount and price: ");
        ioResult = scanf("%d%f", &amount, &price);
        if (ioResult != 2) {
            FLUSH;
            if (ioResult == 1)
                printf(ERR1);
            else
                printf(ERR2);
        } // if
    } while (ioResult != 2);
}
```



# Unformatted I/O

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

**Unformatted I/O**

Direct IO

File Positioning

Module Summary

# Unformatted I/O



# Unformatted I/O: Character I/O

Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

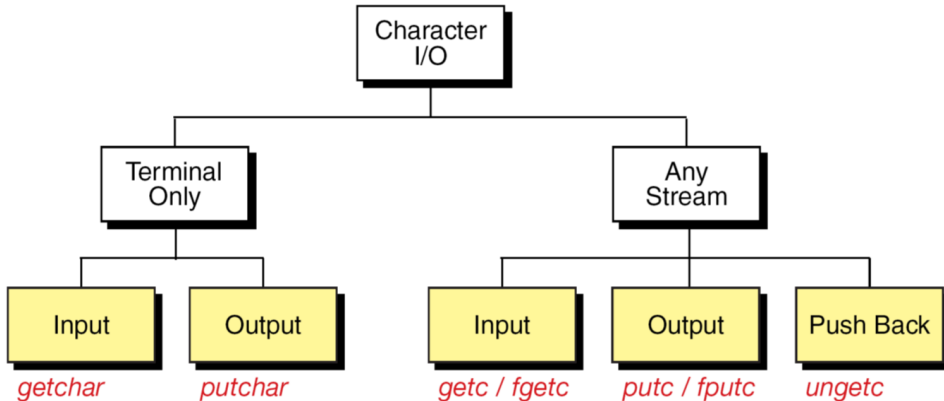
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary





# Create Text File

Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams  
File Open / Close

Formatted I/O  
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

```
/* This program creates a text file */
#include <stdio.h>
int main() {
    FILE* spText;          // Stream
    int c, closeStatus;

    printf("This program copies input to a file.\n");
    printf("When you are through, enter <EOF>.\n\n");

    if (!(spText = fopen("My_New_Text_File.txt", "w"))) {
        printf("Error opening My_New_Text_File.txt for writing");
        return (1);
    } // if open

    while ((c = getchar()) != EOF) // Read characters from stdin. Use ^Z for EOF
        fputc(c, spText);        // Write characters to file

    closeStatus = fclose(spText);
    if (closeStatus == EOF) {
        printf("Error closing file\n");
        return 100;
    } // if

    printf("\n\nYour file is complete\n");
}
```



# Direct Input/Output

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

## Direct Input/Output



# File Write Operation

Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

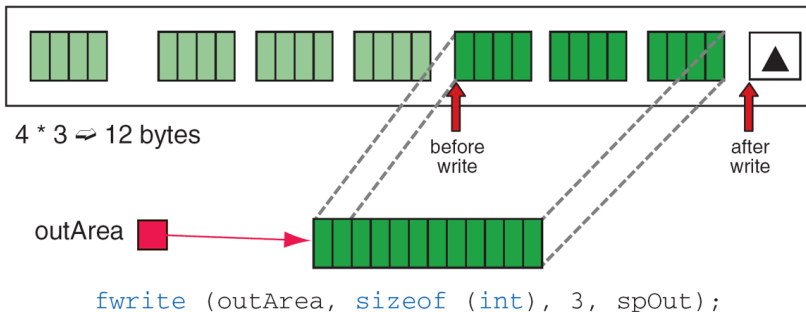
Formatted I/O  
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary





# Writing a Structure

Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

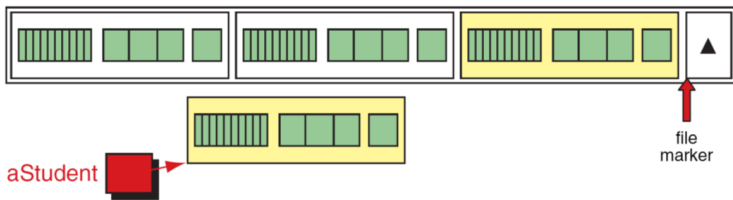
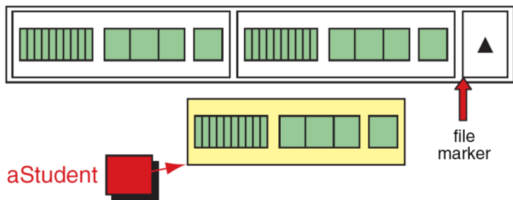
Unformatted I/O

Direct IO

File Positioning

Module Summary

**Before Write**



**After Write**





# File Read Operation

Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

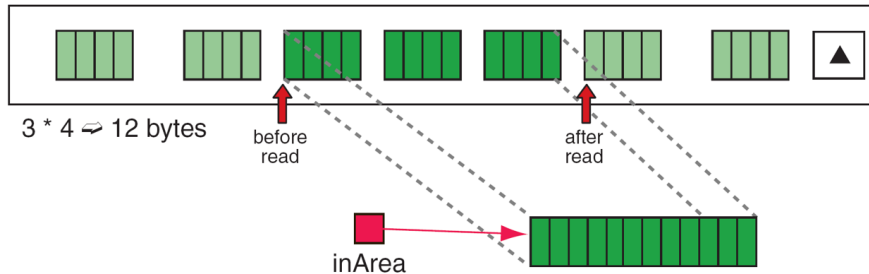
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary



```
fread (inArea, sizeof (int), 3, spData);
```



# Reading a Structure

Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

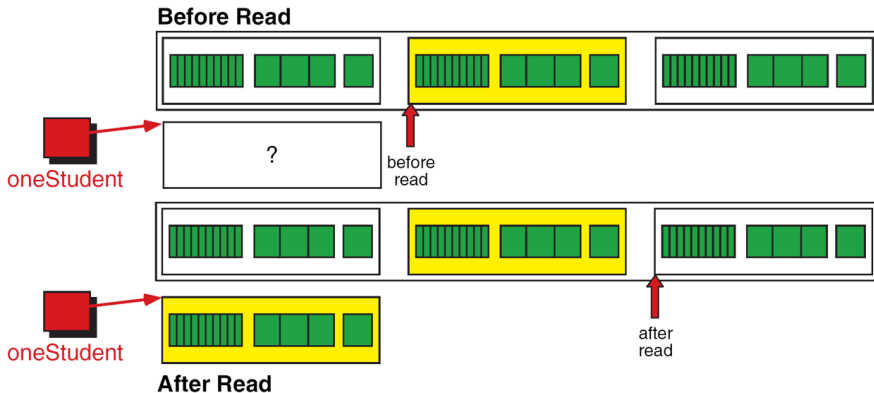
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary





# File Write / Read

## Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

```
size_t fwrite(const void *buffer, size_t size, size_t count, FILE *stream);
```

```
// buffer: pointer to the array where the read objects are stored
```

```
// size: size of each object in bytes
```

```
// count: the number of the objects to be read
```

- Writes count of objects from the given array buffer to the output stream stream. The objects are written as if by reinterpreting each object as an array of `unsigned char` and calling `fputc size` times for each object to write those unsigned chars into stream, in order. The file position indicator for the stream is advanced by the number of characters written
- Returns number of objects written successfully, which may be less than count if an error occurs

```
size_t fread(void *buffer, size_t size, size_t count, FILE *stream);
```

- Reads up to count objects into the array buffer from the given input stream stream as if by calling `fgetc size` times for each object, and storing the results, in the order obtained, into the successive positions of buffer, which is reinterpreted as an array of `unsigned char`. The file position indicator for the stream is advanced by the number of characters read
- Returns number of objects read successfully, which may be less than count if an error or end-of-file condition occurs



# File Positioning

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

Unformatted I/O

Direct IO

**File Positioning**

Module Summary

# File Positioning



# Rewind File

Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

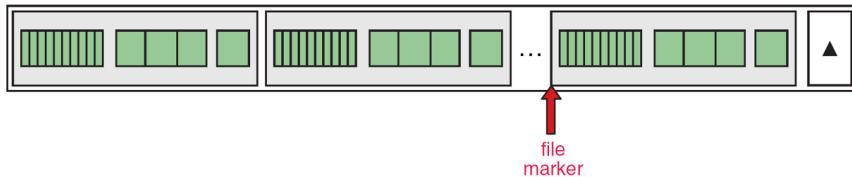
Unformatted I/O

Direct IO

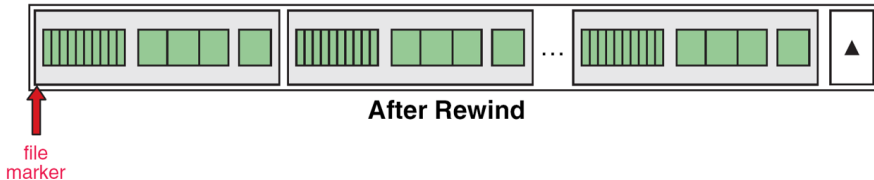
File Positioning

Module Summary

**Before Rewind**



**After Rewind**





# Current Location (ftell) Operation

## Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

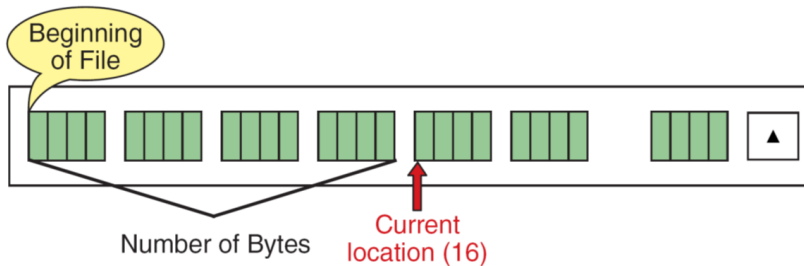
Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary





# File Seek Operation

Module M41

Partha Pratim Das

Weekly Recap

Objectives & Outlines

Standard Library for I/O

Files and Streams

File Open / Close

Formatted I/O

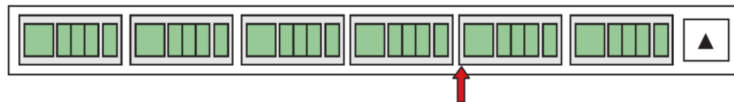
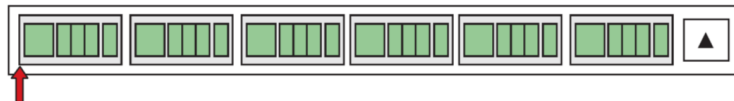
Output  
Read

Unformatted I/O

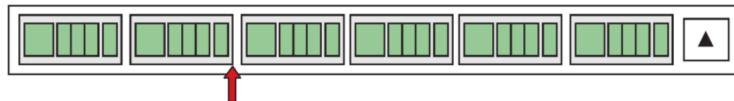
Direct IO

File Positioning

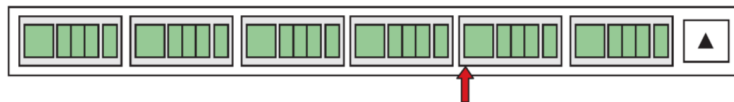
Module Summary



```
fseek (sp, 4 * sizeof(STRUCTURE_TYPE), SEEK_SET);
```



```
fseek (sp, - 4 * sizeof(STRUCTURE_TYPE), SEEK_END);
```



```
fseek (sp, 2 * sizeof(STRUCTURE_TYPE), SEEK_CUR);
```



# File Positioning Function

Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output  
Read

Unformatted I/O

Direct IO

File Positioning

Module Summary

```
long ftell(FILE *stream);
```

- Returns the file position indicator for the file stream `stream` on success or `-1L` if failure occurs
- If the stream is open in binary mode, the value obtained by this function is the number of bytes from the beginning of the file
- If the stream is open in text mode, the value returned by this function is unspecified and is only meaningful as the input to `fseek()`

```
int fseek(FILE *stream, long offset, int origin); // origin=SEEK_SET, SEEK_CUR, or SEEK_END
// offset: number of characters to shift the position relative to origin
// origin: position to which offset is added
```

- Sets the file position indicator for the file stream `stream` to the value pointed to by `offset`. Returns 0 upon success, nonzero value otherwise

```
void rewind(FILE *stream);
```

- Moves the file position indicator to the beginning of the given file stream.
- The function is equivalent to `fseek(stream, 0, SEEK_SET)`, except that `EOF` is cleared

```
int fgetpos(FILE *stream, fpos_t *pos);
int fsetpos(FILE *stream, fpos_t *pos);
```





# Module Summary

## Module M41

Partha Pratim  
Das

Weekly Recap

Objectives &  
Outlines

Standard Library  
for I/O

Files and Streams

File Open / Close

Formatted I/O

Output

Read

Unformatted I/O

Direct IO

File Positioning

**Module Summary**

- Discussed formatted and unformatted I/O using C Standard Library
- Discussed I/O with file and string



## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

# Programming in Modern C++

## Module M42: Input-Output: Streams in C++

Partha Pratim Das

Department of Computer Science and Engineering  
Indian Institute of Technology, Kharagpur

*ppd@cse.iitkgp.ac.in*

*All url's in this module have been accessed in September, 2021 and found to be functional*



# Module Recap

## Module M42

Partha Pratim  
Das

### Objectives & Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- Discussed formatted and unformatted I/O using C Standard Library
- Discussed I/O with file and string



# Module Objectives

## Module M42

Partha Pratim  
Das

### Objectives & Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- To understand object-oriented stream input/output of C++

**Source:** [Chapter 21 - C++ Stream Input/Output](#). © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved.



# Module Outline

## Module M42

Partha Pratim  
Das

### Objectives & Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- 1 Features of C++ I/O
- 2 Streams
- 3 Stream Output
- 4 Stream Input
- 5 File I/O
- 6 Type-safe I/O
- 7 Unformatted I/O
- 8 Stream Manipulators
- 9 Stream States
  - Format States
  - Error States
- 10 Standard I/O Library
- 11 Module Summary

Programming in Modern C++

Partha Pratim Das

M42.4



# Features of C++ I/O

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

## Features of C++ I/O

### Sources:

- Input/output via `<iostream>` and `<cstdio>`, `isocpp`
- Input/Output Library: [cplusplus.com](http://cplusplus.com)
- Input/Output Library: [cppreference.com](http://cppreference.com)
- Chapter 21 - C++ Stream Input/Output. © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved



# Features of C++ I/O

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- Many C++ I/O features are **object-oriented**
  - Use references, function overloading and operator overloading
- C++ uses **type safe** I/O
  - Each I/O operation is automatically performed in a manner sensitive to the data type
- **Extensibility**
  - Users may specify I/O of user-defined types as well as standard types



# Streams

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

**Streams**

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

## Streams

### Sources:

- [Input/output via <iostream> and <cstdio>](#), isocpp
- [Input/Output Library: cplusplus.com](#)
- [Input/Output Library: cppreference.com](#)
- [Chapter 21 - C++ Stream Input/Output](#). © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved





# Streams

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- Stream

- A transfer of information in the form of a sequence of bytes
- The term stream is an abstraction of a construct that allows you to send or receive an unknown number of bytes. The metaphor is a stream of water. You take the data as it comes, or send it as needed. Contrast this to an array, for example, which has a fixed, known length

- I/O Operations

- Input: A stream that flows from an input device (that is, keyboard, disk drive, network connection) to main memory
  - ▷ `istream`
  - ▷ `ifstream`
- Output: A stream that flows from main memory to an output device (that is, screen, printer, disk drive, network connection)
  - ▷ `ostream`
  - ▷ `ofstream`



# Streams

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

**Streams**

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream

Manipulators

Stream States

Format States

Error States

Standard I/O

Library

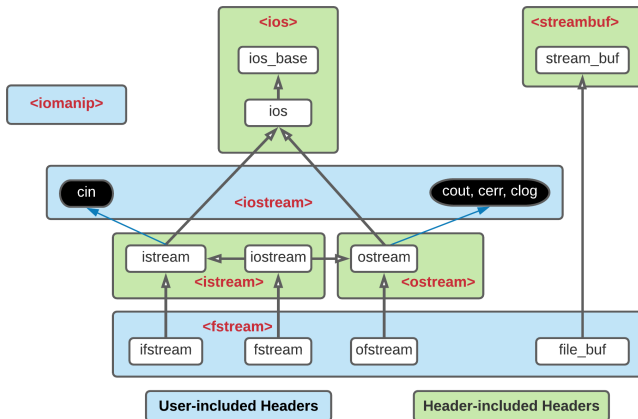
Module Summary

- I/O operations are a bottleneck
  - The time for a stream to flow is many times larger than the time it takes the CPU to process the data in the stream
- Low-level I/O
  - Unformatted
  - Individual byte unit of interest
  - High speed, high volume, but inconvenient for people
- High-level I/O
  - Formatted
  - Bytes grouped into meaningful units: integers, characters, etc.
  - Good for all I/O except high-volume file processing



# <iostream> Header Files

- **iostream** library
  - **<iostream>**: Contains **cin**, **cout**, **cerr** and **clog** objects
  - **<iomanip>**: Contains parameterized stream manipulators





# Stream I/O Classes and Objects

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- **ios:**
  - **istream** and **ostream** inherit from **ios**
    - ▷ **iostream** inherits from **istream** and **ostream**
- **<<** (left-shift operator)
  - Overloaded as stream insertion operator
- **>>** (right-shift operator)
  - Overloaded as stream extraction operator
  - Both operators used with **cin**, **cout**, **cerr** and **clog**, and with user-defined stream objects
- **istream**: input streams
  - **cin >> grade;**
    - ▷ **cin** knows what type of data is to be assigned to **grade** (based on the type of **grade**)
- **ostream**: output streams
  - **cout << grade;**
    - ▷ **cout** knows the type of data to output
  - **cerr << errorMessage;**
    - ▷ **Unbuffered** - prints **errorMessage** immediately
  - **clog << errorMessage;**
    - ▷ **Buffered** - prints **errorMessage** as soon as output buffer is full or flushed



# Stream Output

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

**Stream Output**

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

## Stream Output

### Sources:

- [Input/output via <iostream> and <cstdio>](#), isocpp
- [Input/Output Library: cplusplus.com](#)
- [Input/Output Library: cppreference.com](#)
- [Chapter 21 - C++ Stream Input/Output](#). © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved



# Stream Output

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- **ostream**: performs formatted and unformatted output
  - Uses **put** for characters and **write** for unformatted output
  - Output of integers in decimal, octal and hexadecimal
  - Varying precision for floating points
  - Formatted text outputs
- **<<** is overloaded to output built-in types
  - Can also be used to output user-defined types
  - **cout << '\n';**
    - ▷ Prints newline character
  - **cout << endl;**
    - ▷ **endl** is a stream manipulator that issues a newline character and flushes the output buffer
  - **cout << flush;**
    - ▷ **flush** flushes the output buffer
- **put** member function
  - Outputs one character to specified stream: **cout.put('A');**
  - Returns a reference to the object that called it, so may be *cascaded*: **cout.put('A').put('\n');**
  - May be called with an ASCII-valued expression: **cout.put(65);**
    - ▷ Outputs A



# Print Built-in Type Data

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream

Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

```
#include <iostream>
using namespace std;
int main() {
    int i = 17; long l = 0x012a78cb; // 19560651
    long long unsigned int i64 = 0x012a78cb2597ac3d; // 84012356964166717
    float f = 15.0 / 7; double d = 15.0 / 7;
    char c = 'x'; const char *s = "ppd";
    int *p = &i;

    cout << i << " ";           // int      // 17 Optional dec may be used
    cout << hex << i << endl;    // hex      // 11
    cout << oct << i << endl;    // oct      // 21
    cout << l << " ";           // long     // 19560651
    cout << i64 << " ";         // int 64   // 84012356964166717
    cout << f << " ";           // float    // 2.14286
    cout << d << " ";           // double   // 2.14286
    cout << c << " ";           // char     // x
    cout << s << " ";           // string   // ppd
    cout << (void*)(s) << endl; // pointer  // 0x55c825222009 // Address of 1st character of the string
    cout << p << " ";           // pointer  // 0x7fff9a17cf68
}
```

- An integer (**int**) may be printed in decimal (**dec**, by default), octal (**oct**) or hexadecimal (**hex**) format
- A **char\*** pointer prints the string. To print the pointer value, cast to **void\*** by **static\_cast<const void\*>** or **(void\*)**



# Print User-defined Type Data

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

```
#include <iostream>
using namespace std;

class Complex {
    double re, im; // Encapsulated
public:
    Complex(double r, double i) : re(r), im(i) { }

    // UDT Specific print function
    friend ostream& operator<<(ostream& os, const Complex& c) {
        cout << "(" << c.re << ", " << c.im << ")";
        return os;
    }
};

int main() {
    Complex c1 = { 2.5, 7.3 }, c2(4.3, 8.9);

    cout << c1 << " "; << c2 << endl; // Cascading the printing
}
```

(2.5, 7.3); (4.3, 8.9)





# Print Built-in Type Data: C vis-a-vis C++

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

```
#include <cstdio>
#include <iostream>
using namespace std;
int main() {
    int i = 17;
    long l = 0x012a78cb; // 19560651
    long long unsigned int i64 = 0x012a78cb2597ac3d; // 84012356964166717
    float f = 15.0 / 7;
    double d = 15.0 / 7;
    char c = 'x';
    const char *s = "ppd";
    int *p = &i;
    cout << i << " ";          printf("%d\n", i);          // dec          // 17 17 Opt. dec may be used in C++
    cout << hex << i << endl;    printf("%x\n", i);          // hex          // 11 11
    cout << oct << i << endl;    printf("%o\n", i);          // oct          // 21 21
    cout << l << " ";           printf("%ld\n", l);          // long         // 19560651 19560651
    cout << i64 << " ";         printf("%llu\n", i64);       // int 64       // 84012356964166717 84012356964166717
    cout << f << " ";           printf("%f\n", f);          // float        // 2.14286 2.142857
    cout << d << " ";           printf("%lf\n", d);         // double       // 2.14286 2.142857
    cout << c << " ";           printf("%c\n", c);          // char         // x x
    cout << s << " ";           printf("%s\n", s);          // string       // ppd ppd
    cout << p << " ";           printf("%p\n", p);          // pointer      // 0x7ffc28102988 0x7ffc28102988
}
```

- Note the use of **hex** and **oct** in C++ and the difference in default precision for **float** and **double** between C++ and C



# Stream Input

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

**Stream Input**

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

## Stream Input

### Sources:

- Input/output via `<iostream>` and `<cstdio>`, isocpp
- Input/Output Library: [cplusplus.com](http://cplusplus.com)
- Input/Output Library: [cppreference.com](http://cppreference.com)
- Chapter 21 - C++ Stream Input/Output. © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved



# Stream Input

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- **>>** (stream-extraction)
  - Used to perform stream input
  - Normally ignores whitespaces (spaces, tabs, newlines)
  - Returns zero (**false**) when **EOF** is encountered, otherwise returns reference to the object from which it was invoked (that is, **cin**)
- **>>** controls the state bits of the stream
  - **endl** is a stream manipulator that issues a newline character and flushes the output buffer
- **>>** and **<<** have relatively high precedence
  - Conditional and arithmetic expressions must be contained in parentheses
- Common way to perform loops

```
while (cin >> grade)
```

  - Extraction returns 0 (**false**) when **EOF** encountered, and loop ends



# Member Functions

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- `cin.eof()`
  - returns true if end-of-file has occurred on `cin`
- `cin.get()`
  - inputs a character from stream (even white spaces) and returns it
- `cin.get(c)`
  - inputs a character from stream and stores it in `c`
- `cin.get(array, size)`
  - Accepts 3 arguments: array of characters, the size limit, and a delimiter (default of `'\n'`)
  - Uses the array as a buffer
  - When the delimiter is encountered, it remains in the input stream
  - Null character is inserted in the array
  - Unless delimiter flushed from stream, it will stay there



# Member Functions

## Module M42

Partha Pratim Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- `cin.getline(array, size)`
  - Operates like `cin.get(buffer, size)` but it discards the delimiter from the stream and does not store it in array
  - Null character inserted into array
- `ignore`
  - Operates like `cin.get(buffer, size)` but it discards the delimiter from the stream and does not store it in array
  - Null character inserted into array
- `putback`
  - Places the previous character obtained by get back in to the stream
- `peek`
  - Returns the next character from the stream without removing it



# File I/O

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

## File I/O

### Sources:

- Input/output via `<iostream>` and `<cstdio>`, `isocpp`
- Input/Output Library: [cplusplus.com](http://cplusplus.com)
- Input/Output Library: [cppreference.com](http://cppreference.com)
- Chapter 21 - C++ Stream Input/Output. © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved



# Input / Output with Files

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- Open

- Like in C, files need to be first opened and associated with a stream

```
ofstream myfile;           // Output stream
myfile.open("example.txt"); // Open: Associate file example.txt to output stream myfile
-----
ofstream myfile("example.txt"); // Output stream opened and associated
myfile.is_open(); // Check if open has worked correctly
```

- Unlike C (where stream is a pointer), stream is an object in C++
- Unlike C (where mode is specified by a string flag), stream object itself is of i/p or o/p types

- Read / Write

- Like in C, we perform formatted or unformatted I/O on an open stream (file)
- Unlike C (where functions for formatted I/O are variadic and needs explicit format specification), objects are read / written using streaming operators for the data types

- Close

- Like in C, streams need to be closed when done and disassociated from the file
- ```
myfile.close(); // Close: Flush stream to file and disassociate from stream
```

- Binary Files

- Use `ios::binary` flag in the opening mode



# Input / Output with Files

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

```
// Writing to Output File
```

```
#include <iostream>
```

```
#include <fstream>
```

```
using namespace std;
```

```
int main () { ofstream myfile;
```

```
    myfile.open("example.txt");
```

```
    myfile << "Writing this to a file.\n";
```

```
    myfile.close();
```

```
}
```

```
// Output stream
```

```
// Open: Associate file example.txt to output stream myfile
```

```
// Stream to output
```

```
// Close: Flush stream to file and disassociate from stream
```

```
// Reading from Input File
```

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main () { ifstream myfile("example.txt"); // Input stream
```

```
    string line;
```

```
    if (myfile.is_open()) {
```

```
        while (getline(myfile, line))
```

```
            cout << line << '\n';
```

```
        myfile.close();
```

```
    }
```

```
    else cout << "Unable to open file";
```

```
}
```

```
// Open: Associate file example.txt to input stream myfile
```

```
// Unformatted Read: Get by line from stream
```

```
// Close: Disassociate file from stream
```





# Type-safe I/O

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

## Type-safe I/O

### Sources:

- Input/output via `<iostream>` and `<cstdio>`, `isocpp`
- Input/Output Library: [cplusplus.com](http://cplusplus.com)
- Input/Output Library: [cppreference.com](http://cppreference.com)
- Chapter 21 - C++ Stream Input/Output. © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved



# Type-safe I/O

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- << and >> operators
  - Gets format from type of data being read / written
  - Overloaded to accept data of different types
  - Cascading for ease of expression
  - Avoids the use of error-prone variadic functions
  - When unexpected data encountered, error flags set
  - Program stays in control



# Type-safe I/O: C vis-a-vis C++

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

```
#include <cstdio>
#include <iostream>
using namespace std;

int main() {
    int i = 5, j = 3;
    double d = 2.37483;

    // C I/O is type-unsafe
    printf("%d %d\n", i, j); // Okay: 5 3
    printf("%d\n", i, j);    // Error. Missing format spec. Prints garbage for j: 5 2757403
    printf("%d %d\n", i);    // Error. Missing second value. Ignored: 5

    printf("%lf\n", d);      // Okay: 2.374830
    printf("%d\n", d);      // Error. Wrong integer format for double value: -553878982
    printf("%lf\n", i);      // Error. Wrong double format for integer value: 0.000000

    // C++ I/O is type-safe
    cout << i << ' ';
    cout << j << ' ';
    cout << d << endl;      // Okay: 5 3 2.37483
}
```



# Type-safe I/O: User-defined Operators

Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

// Discussed in **Module 19: Program 19.06**

```
#include <iostream>
```

```
using namespace std;
```

```
class Complex { double re, im; // Encapsulated
```

```
public: Complex(double r = 0.0, double i = 0.0) : re(r), im(i) { }
```

```
    friend ostream& operator<<(ostream& os, const Complex& c) { // UDT Specific print function
```

```
        cout << "(" << c.re << ", " << c.im << ")";
```

```
        return os;
```

```
    }
```

```
    friend istream& operator>>(istream& os, Complex& c) { // UDT Specific scan function
```

```
        cin >> c.re >> c.im;
```

```
        return os;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Complex c1 = { 2.5, 7.3 }, c2(4.3, 8.9), c3, c4;
```

```
    cout << c1 << "; " << c2 << endl; // Cascading the printing: (2.5, 7.3); (4.3, 8.9)
```

```
    cout << c3 << "; " << c4 << endl; // Cascading the printing: (0, 0); (0, 0)
```

```
    cin >> c3 >> c4; // Cascading the scanning: 1.2 3.7 3.4 9.6
```

```
    cout << c3 << "; " << c4 << endl; // Cascading the printing: (1.2, 3.7); (3.4, 9.6)
```

```
}
```



# Unformatted I/O

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

## Unformatted I/O

### Sources:

- Input/output via `<iostream>` and `<cstdio>`, `isocpp`
- Input/Output Library: [cplusplus.com](http://cplusplus.com)
- Input/Output Library: [cppreference.com](http://cppreference.com)
- Chapter 21 - C++ Stream Input/Output. © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved



# Unformatted I/O

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- `read` and `write` member functions
  - Unformatted I/O
  - Input/output raw bytes to or from a character array in memory
  - Since the data is unformatted, the functions will not terminate at a newline character for example
  - Instead, like `getline`, they continue to process a designated number of characters
  - If fewer than the designated number of characters are read, then the `failbit` is set
- `gcount`
  - Returns the total number of characters read in the last input operation



# Stream Manipulators

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

## Stream Manipulators

### Sources:

- [Input/output via <iostream> and <cstdio>](#), isocpp
- [Input/Output Library: cplusplus.com](#)
- [Input/Output Library: cppreference.com](#)
- [Chapter 21 - C++ Stream Input/Output](#). © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved



# Functionality of Stream Manipulators

## Module M42

Partha Pratim Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

**Stream  
Manipulators**

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- Setting field widths
- Setting precisions
- Setting and unsetting format flags
- Setting the fill character in fields
- Flushing streams
- Inserting a newline in the output stream and flushing the stream
- Inserting a null character in the output stream and skipping whitespace in the input stream





# Integral Stream Base

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- **dec** (default), **oct** or **hex**
  - Change base of which integers are interpreted from the stream

```
int n = 15;
cout << hex << n;
```
  - Prints "F"
- **setbase**:
  - Changes base of integer output
  - Load `<iomanip>`
  - Accepts an integer argument (10, 8, or 16)

```
cout << setbase(16) << n;
```
  - Parameterized stream manipulator - takes an argument



# Floating-Point Precision

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- **precision**

- Member function
- Sets number of digits to the right of decimal point

```
cout.precision(2);
```

- `cout.precision()` returns current precision setting

- **setprecision:**

- Parameterized stream manipulator
- Like all parameterized stream manipulators, `<iomanip>` required
- Specify precision

```
cout << setprecision(2) << x;
```

- For both methods, changes last until a different value is set



# Field Width

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- `ios width` member function
  - Sets field width (number of character positions a value should be output or number of characters that should be input)
  - Returns previous width
  - If values processed are smaller than width, fill characters inserted as padding
  - Values are not truncated - full number printed

```
cin.width(5);
```

- `setw` stream manipulator

```
cin >> setw(5) >> string;
```
- Remember to reserve one space for the null character



# User-Defined Manipulators

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

**Stream  
Manipulators**

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- We can create our own stream manipulators
  - `bell`
  - `ret` (carriage return)
  - `tab`
  - `endLine`
- Parameterized stream manipulators
  - Consult installation manuals



# Stream States

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

**Stream States**

Format States

Error States

Standard I/O  
Library

Module Summary

## Stream States

### Sources:

- [Input/output via <iostream> and <cstdio>](#), isocpp
- [Input/Output Library: cplusplus.com](#)
- [Input/Output Library: cppreference.com](#)
- [Chapter 21 - C++ Stream Input/Output](#). © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved



# Stream Format States

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

**Format States**

Error States

Standard I/O  
Library

Module Summary

- Format flags
  - Specify formatting to be performed during stream I/O operations
- `setf`, `unsetf` and `flags`
  - Member functions that control the flag settings



# Format State Flags

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- Format State Flags
  - Defined as an enumeration in class `ios`
  - Can be controlled by member functions
  - `flags` - specifies a value representing the settings of all the flags
  - Returns long value containing prior options
  - `setf` - one argument, "ors" flags with existing flags
  - `unsetf` - `unsets` flags
  - `setiosflags` - parameterized stream manipulator used to set flags
  - `resetiosflags` - parameterized stream manipulator, has same functions as `unsetf`
- Flags can be combined using bitwise `OR ( | )`



# Trailing Zeros and Decimal Points

## Module M42

Partha Pratim Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- `ios::showpoint`

- Forces a float with an integer value to be printed with its decimal point and trailing zeros

```
cout.setf(ios::showpoint)
cout << 79;
```

79 will print as 79.00000

- ▷ Number of zeros determined by precision settings





# Justification

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- `ios::left`
  - Fields left-justified with padding characters to the right
- `ios::right`
  - Default setting
  - Fields right-justified with padding characters to the left
- Character used for padding set by
  - `fill` member function
  - `setfill` parameterized stream manipulator
  - Default character is space
- `internal` flag
  - Number's sign left-justified
  - Number's magnitude right-justified
  - Intervening spaces padded with the fill character
- `static` data member `ios::adjustfield`
  - Contains `left`, `right` and `internal` flags
  - Number's magnitude right-justified
  - `ios::adjustfield` must be the second argument to `setf` when setting the `left`, `right` or `internal` justification flags `cout.setf(ios::left, ios::adjustfield);`



# Padding

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- `fill`
  - Specifies the fill character
  - Space is default
  - Returns the prior padding character

```
cout.fill('*');
```

- `setfill` manipulator

- Also sets fill character

```
cout << setfill ('*');
```



# Integral Stream Base

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- `ios::basefield` static member
  - Used similarly to `ios::adjustfield` with `setf`
  - Includes the `ios::oct`, `ios::hex` and `ios::dec` flag bits
  - Specify that integers are to be treated as octal, hexadecimal and decimal values
  - Default is decimal
  - Default for stream extractions depends on form inputted
    - ▷ Integers starting with `0` are treated as *octal*
    - ▷ Integers starting with `0x` or `0X` are treated as *hexadecimal*
  - Once a base specified, settings stay until changed



# Floating-Point Numbers / Scientific Notation

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- `ios::scientific`

- Forces output of a floating point number in scientific notation:

`1.946000e+009`

- `ios::fixed`

- Forces floating point numbers to display a specific number of digits to the right of the decimal (specified with `precision`)

- `static` data member `ios::floatfield`

- Contains `ios::scientific` and `ios::fixed`
- Used similarly to `ios::adjustfield` and `ios::basefield` in `setf` like `cout.setf(ios::scientific, ios::floatfield);`
- `cout.setf(0, ios::floatfield)` restores default format for outputting floating-point numbers



# Uppercase/Lowercase Control

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- `ios::uppercase`

- Forces uppercase `E` to be output with scientific notation

`4.32E+010`

- Forces uppercase `X` to be output with hexadecimal numbers, and causes all letters to be uppercase

`75BDE`



# Setting and Resetting the Format Flags

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream

Manipulators

Stream States

Format States

Error States

Standard I/O

Library

Module Summary

- **flags**
  - Without argument, returns the current settings of the format flags (as a **long** value)
  - With a **long** argument, sets the format flags as specified
    - ▷ Returns prior settings
- **setf**
  - Sets the format flags provided in its argument
  - Returns the previous flag settings as a **long** value
  - Unset the format using **unsetf** member function as

```
long previousFlagSettings =  
cout.setf(ios::showpoint | ios::showpos);
```
- **setf** with two **long** arguments `cout.setf(ios::left, ios::adjustfield);` clears the bits of **ios::adjustfield** then sets **ios::left**
  - This version of **setf** can be used with
  - `ios::basefield(ios::dec, ios::oct, ios::hex)`
  - `ios::floatfield(ios::scientific, ios::fixed)`
  - `ios::adjustfield (ios::left, ios::right, ios::internal)`
- **unsetf**
  - Resets specified flags
  - Returns previous settings



# Stream Error States

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

**Error States**

Standard I/O Library

Module Summary

- `eofbit`
  - Set for an input stream after end-of-file encountered
  - `cin.eof()` returns true if end-of-file has been encountered on `cin`
- `failbit`
  - Set for a stream when a format error occurs
  - `cin.fail()` - returns true if a stream operation has failed
  - Normally possible to recover from these errors



# Stream Error States

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- `badbit`
  - Set when an error occurs that results in data loss
  - `cin.bad()` returns true if stream operation failed
  - normally nonrecoverable
- `goodbit`
  - Set for a stream if neither `eofbit`, `failbit` or `badbit` are set
  - `cin.good()` returns `true` if the `bad`, `fail` and `eof` functions would all return `false`
  - I/O operations should only be performed on “good” streams
- `rdstate`
  - Returns the state of the stream
  - Stream can be tested with a switch statement that examines all of the state bits
  - Easier to use `eof`, `bad`, `fail`, and `good` to determine state





# Stream Error States

## Module M42

Partha Pratim Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- `clear`
  - Used to restore a stream's state to "good"
  - `cin.clear()` clears `cin` and sets `goodbit` for the stream
  - `cin.clear(ios::failbit)` actually sets the `failbit`
    - ▷ Might do this when encountering a problem with a user-defined type
- Other operators
  - `operator!`
    - ▷ Returns true if `badbit` or `failbit` set
  - `operator void*`
    - ▷ Returns false if `badbit` or `failbit` set
  - Useful for file processing



# Standard I/O Library

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

## Standard I/O Library

### Sources:

- Input/output via `<iostream>` and `<cstdio>`, `isocpp`
- Input/Output Library: [cplusplus.com](http://cplusplus.com)
- Input/Output Library: [cppreference.com](http://cppreference.com)
- Chapter 21 - C++ Stream Input/Output. © Copyright 1992–2004 by Deitel & Associates, Inc. and Pearson Education Inc. All Rights Reserved



# Library Organization

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

Stream States

Format States

Error States

Standard I/O Library

Module Summary

- `<ios>`, `<istream>`, `<ostream>`, `<streambuf>` and `<iosfwd>` are not usually included directly in most C++ programs. They describe the base classes of the hierarchy and are automatically included by other header files of the library that contain the derived classes.
- `<iostream>` declares the objects used to communicate through the standard input and output (including `cin` and `out`)
- `<fstream>` defines the file stream classes (like template `basic_ifstream` or class `ofstream`) as well as the internal buffer objects used (`basic_filebuf`). These classes are used to manipulate files with streams.
- `<sstream>`. The classes defined in this file are used to manipulate STL string objects as if they were streams.
- `<iomanip>` declares some standard manipulators with parameters to be used with extraction and insertion operators to modify internal flags and formatting options.



# Input-Output Class Hierarchy

Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

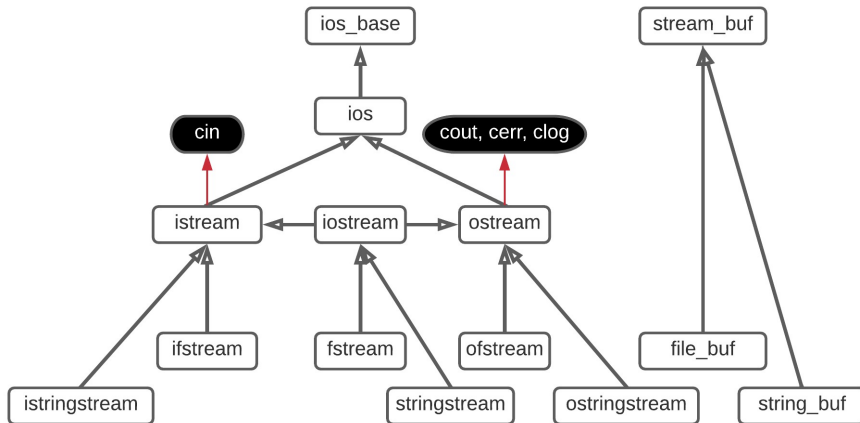
Stream States

Format States

Error States

Standard I/O Library

Module Summary



Sources: [Input/Output Library: cplusplus.com](http://cplusplus.com), [Input/Output Library: cppreference.com](http://cppreference.com)



# Input-Output Classes in Header Files

## Module M42

Partha Pratim Das

Objectives & Outlines

Features of C++ I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream Manipulators

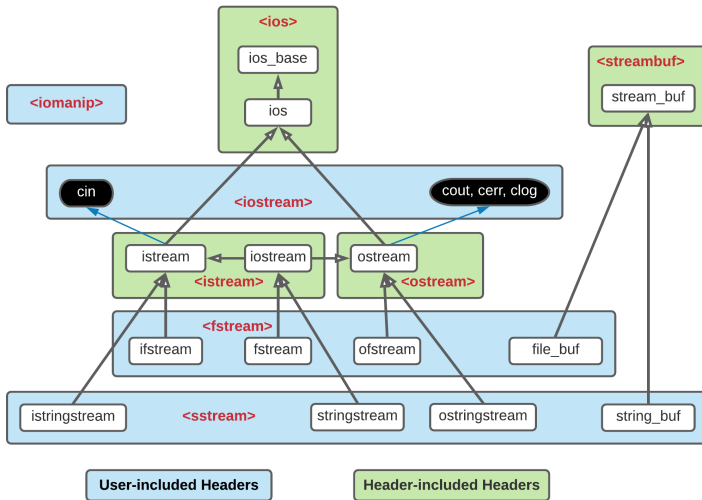
Stream States

Format States

Error States

Standard I/O Library

Module Summary



Sources: [Input/Output Library: cplusplus.com](http://cplusplus.com), [Input/Output Library: cppreference.com](http://cppreference.com)

Programming in Modern C++

Partha Pratim Das

M42.52



# Header Organization

## Module M42

Partha Pratim Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- `<iostream>`
  - `<istream>`
    - ▷ `<ostream>`
      - `<ios>`
- `<fstream>`
  - `<istream>`
- `<sstream>`
  - `<string>`
- `<iomanip>`
  - `<istream>`
- `<streambuf>`
  - `<xiosbase>`
- `<xiosbase>`
- `<iosfwd>`



# Module Summary

## Module M42

Partha Pratim  
Das

Objectives &  
Outlines

Features of C++  
I/O

Streams

Stream Output

Stream Input

File I/O

Type-safe I/O

Unformatted I/O

Stream  
Manipulators

Stream States

Format States

Error States

Standard I/O  
Library

Module Summary

- Understood object-oriented I/O of C++
- Learnt the major standard library components



## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

# Programming in Modern C++

## Module M43: C++ Standard Library (Generic Programming): Part 1

Partha Pratim Das

Department of Computer Science and Engineering  
Indian Institute of Technology, Kharagpur

*ppd@cse.iitkgp.ac.in*

*All url's in this module have been accessed in September, 2021 and found to be functional*





# Module Recap

## Module M43

Partha Pratim  
Das

### Objectives & Outlines

#### Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

#### Generic Programming

Common Tasks

Lifting Example

Model

Examples

#### Module Summary

- Understood object-oriented I/O of C++
- Learnt the major standard library components



# Module Objectives

## Module M43

Partha Pratim  
Das

### Objectives & Outlines

#### Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

#### Generic Programming

Common Tasks

Lifting Example

Model

Examples

#### Module Summary

- To get an overview of Standard Library components of C++
- To understand generic programming for STL



# Module Outline

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

- 1 Standard Library
  - C Standard Library
  - C++ Standard Library
    - std
    - Header Conventions

- 2 Generic Programming
  - Common Tasks
  - Lifting Example
  - Algorithms-Iterators-Containers Model
  - Examples

- 3 Module Summary



# Standard Library

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

## Standard Library

### Sources:

- [Standard library](#), Wikipedia
- [C math.h library functions](#)



# What is Standard Library?

## Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

- A *standard library in programming* is the library *made available across implementations of a language*
- These libraries are usually described in *language specifications (C/C++)*; however, they may also be determined (in part or whole) *by informal practices of a language's community (Python)*
- A language's standard library is *often treated as part of the language by its users*, although the *designers may have treated it as a separate entity*
- Many language specifications define a *core set that must be made available in all implementations*, in addition to *other portions which may be optionally implemented*
- The line between a *language and its libraries* therefore *differs from language to language*
- Bjarne Stroustrup, designer of C++, writes:

*What ought to be in the standard C++ library? One ideal is for a programmer to be able to find every interesting, significant, and reasonably general class, function, template, etc., in a library. However, the question here is not, "What ought to be in some library?" but "What ought to be in the standard library?" The answer "Everything!" is a reasonable first approximation to an answer to the former question but not the latter. A standard library is something every implementer must supply so that every programmer can rely on it.*

- This suggests a *relatively small standard library*, containing only the constructs that *"every programmer" might reasonably require when building a large collection of software*
- **This is the philosophy that is used in the C and C++ standard libraries**

Source: [Standard library, Wiki](#)

Programming in Modern C++

Partha Pratim Das

M43.6



# C Standard Library: Common Library Components

## Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

| Component             | Data Types, Manifest Constants, Macros, Functions, ...                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>stdio.h</code>  | Formatted and un-formatted file input and output including functions <ul style="list-style-type: none"><li>• <code>printf</code>, <code>scanf</code>, <code>fprintf</code>, <code>fscanf</code>, <code>sprintf</code>, <code>sscanf</code>, <code>feof</code>, etc.</li></ul>                                                                                                                                   |
| <code>stdlib.h</code> | Memory allocation, process control, conversions, pseudo-random numbers, searching, sorting <ul style="list-style-type: none"><li>• <code>malloc</code>, <code>free</code>, <code>exit</code>, <code>abort</code>, <code>atoi</code>, <code>strtold</code>, <code>rand</code>, <code>bsearch</code>, <code>qsort</code>, etc.</li></ul>                                                                          |
| <code>string.h</code> | Manipulation of C strings and arrays <ul style="list-style-type: none"><li>• <code>strcat</code>, <code>strcpy</code>, <code>strcmp</code>, <code>strlen</code>, <code>strtok</code>, <code>memcpy</code>, <code>memmove</code>, etc.</li></ul>                                                                                                                                                                 |
| <code>math.h</code>   | Common mathematical operations and transformations <ul style="list-style-type: none"><li>• <code>cos</code>, <code>sin</code>, <code>tan</code>, <code>acos</code>, <code>asin</code>, <code>atan</code>, <code>exp</code>, <code>log</code>, <code>pow</code>, <code>sqrt</code>, etc.</li></ul>                                                                                                               |
| <code>errno.h</code>  | Macros for reporting and retrieving error conditions through error codes stored in a static memory location called <code>errno</code> <ul style="list-style-type: none"><li>• <code>EDOM</code> (parameter outside a function's domain – <code>sqrt(-1)</code>),</li><li>• <code>ERANGE</code> (result outside a function's range), or</li><li>• <code>EILSEQ</code> (an illegal byte sequence), etc.</li></ul> |

*A header file typically contains manifest constants, macros, necessary struct / union types, typedef's, function prototype, etc.*



# C Standard Library: math.h

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

```
/* math.h
 * This file has no copyright assigned and is placed in the Public Domain.
 * This file is a part of the mingw-runtime package.
 * Mathematical functions.
 */
#ifndef _MATH_H_
#define _MATH_H_
#ifndef __STRICT_ANSI__ // conditional exclusions for ANSI
// ...
#define M_PI 3.14159265358979323846 // manifest constant for pi
// ...
struct _complex { // struct of _complex type
    double      x;      /* Real part */
    double      y;      /* Imaginary part */
};
_CRTIMP double __cdecl _cabs (struct _complex); // cabs(.) function header
// ...
#endif /* __STRICT_ANSI__ */
// ...
_CRTIMP double __cdecl sqrt (double); // sqrt(.) function header
// ...
#define isfinite(x) ((fpclassify(x) & FP_NAN) == 0) // macro isfinite(.) to check if a number is finite
// ...
#endif /* _MATH_H_ */
```

Source: [C math.h library functions](#)  
Programming in Modern C++



# C++ Standard Library: Common Library Components

## Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

| Component                        | Data Types, Manifest Constants, Macros, Functions, Classes, ...                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>iostream</code>            | Stream input and output for standard I/O • <code>cout</code> , <code>cin</code> , <code>endl</code> , ..., etc.                                                                                                                                                                                                                                                                                                                                       |
| <code>fstream</code>             | (Module 42)                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <code>string</code>              | Manipulation of string objects • Relational operators, IO operators, Iterators, etc.                                                                                                                                                                                                                                                                                                                                                                  |
| <code>memory</code>              | High-level memory management • Pointers: <code>unique_ptr</code> , <code>shared_ptr</code> , <code>weak_ptr</code> , & <code>allocator</code> etc.                                                                                                                                                                                                                                                                                                    |
| <code>exception</code>           | Generic Error Handling • <code>exception</code> , <code>bad_exception</code> , <code>unexpected_handler</code> , & <code>terminate_handler</code>                                                                                                                                                                                                                                                                                                     |
| <code>stdexcept</code>           | Standard Error Handling • <code>logic_error</code> , <code>invalid_argument</code> , <code>domain_error</code> , <code>length_error</code> , <code>out_of_range</code> , <code>runtime_error</code> , <code>range_error</code> , <code>overflow_error</code> , <code>underflow_error</code> , etc.                                                                                                                                                    |
| STL                              | Utilities                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Containers</b>                | <code>vector</code> , <code>deque</code> , <code>list</code> , <code>stack</code> , <code>queue</code> , <code>priority_queue</code> , <code>set</code> , <code>multiset</code> , <code>map</code> , <code>multimap</code><br>C++11: <code>array</code> , <code>forward_list</code> , <code>unordered_set/multiset/map/multimap</code>                                                                                                                |
| <b>Iterators</b>                 | <code>begin</code> & <code>end</code> , <code>rbegin</code> & <code>rend</code> . C++11: <code>cbegin</code> & <code>cend</code> , <code>crbegin</code> & <code>crend</code> ,                                                                                                                                                                                                                                                                        |
| <code>algorithm</code>           | <b>Non-Numerical:</b> <code>for_each</code> , <code>find</code> , <code>find_if</code> , <code>count</code> , <code>search</code> , <code>copy</code> , <code>move</code> , <code>swap</code> , <code>replace</code> , <code>fill</code> , <code>generate</code> , <code>remove</code> , <code>reverse</code> , <code>rotate</code> , <code>sort</code> , <code>binary_search</code> , <code>merge</code> , <code>min</code> , <code>max</code> , ... |
| <code>numeric</code>             | <b>Numerical:</b> <code>accumulate</code> , <code>adjacent_difference</code> , <code>inner_product</code> , and <code>partial_sum</code> . C++11: <code>iota</code>                                                                                                                                                                                                                                                                                   |
| <b>Functions</b>                 | <code>equal_to</code> , <code>not_equal_to</code> , <code>greater</code> , <code>greater_equal</code> , <code>less</code> , <code>less_equal</code> ; <code>plus</code> , <code>minus</code> , <code>multiplies</code> , <code>divides</code> , <code>modulus</code> ; <code>logical_and</code> , <code>logical_not</code> , <code>logical_or</code> . C++11: <code>bit_and</code> , <code>bit_or</code> , <code>bit_xor</code>                       |
| Imported from C Standard Library |                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <code>cmath</code>               | Common mathematical operations and transformations<br>• <code>cos</code> , <code>sin</code> , <code>tan</code> , <code>acos</code> , <code>asin</code> , <code>atan</code> , <code>exp</code> , <code>log</code> , <code>pow</code> , <code>sqrt</code> , etc.                                                                                                                                                                                        |
| <code>cstdlib</code>             | Memory alloc., process control, conversions, pseudo-rand nos., searching, sorting<br>• <code>malloc</code> , <code>free</code> , <code>exit</code> , <code>abort</code> , <code>atoi</code> , <code>strtold</code> , <code>rand</code> , <code>bsearch</code> , <code>qsort</code> , etc.                                                                                                                                                             |





# namespace std for C++ Standard Library

## Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

**std**

Header Conventions

Generic

Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

## C Standard Library

- All names are global
- `stdout`, `stdin`, `printf`, `scanf`

### W/o using

```
#include <iostream>

int main() {

    std::cout << "Hello World in C++"
               << std::endl;

    return 0;
}
```

## C++ Standard Library

- All names are within `std namespace`
- `std::cout`, `std::cin`
- Use `using namespace std;`

to get rid of writing `std::` for every standard library name

### W/ using

```
#include <iostream>
using namespace std;

int main() {

    cout << "Hello World in C++"
         << endl;

    return 0;
}
```



# Standard Library: C/C++ Header Conventions

Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

|             | C Header                                                                                                                                 | C++ Header                                                              |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| C Program   | Use <code>.h</code> . Example: <code>#include &lt;stdio.h&gt;</code><br><i>Names in global namespace</i>                                 | Not applicable                                                          |
| C++ Program | Prefix <code>c</code> , no <code>.h</code> . Example: <code>#include &lt;cstdio&gt;</code><br><i>Names in <code>std</code> namespace</i> | No <code>.h</code> . Example:<br><code>#include &lt;iostream&gt;</code> |

- A C std. library header is used in C++ with prefix '`c`' and without the `.h`. These are in `std` namespace:

```
#include <cmath> // In C it is <math.h>
...
```

```
std::sqrt(5.0); // Use with std::
```

It is possible that a C++ program include a C header as in C. Like:

```
#include <math.h> // Not in std namespace
...
sqrt(5.0);        // Use without std::
```

This, however, is not preferred

- **Using `.h` with C++ header files, like `iostream.h`, is disastrous. These are deprecated. It is dangerous, yet true, that some compilers do not error out on such use. Exercise caution.**



# Generic Programming

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

## Generic Programming

### Source:

- [GENERIC PROGRAMMING](#), Sean Parent, code::dive conference 2018
- [Chapter 20 The STL \(containers, iterators, and algorithms\)](#), Bjarne Stroustrup
- [Chapter 21 The STL \(maps and algorithms\)](#), Bjarne Stroustrup



# Common Programming Tasks

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

- Collect data into containers
- Organize data
  - For printing
  - For fast access
- Retrieve data items
  - By index (for example, get the  $N^{th}$  element)
  - By value (for example, get the first element with the value `Chocolate`)
  - By properties (for example, get the first elements where `age < 64`)
- Add data
- Remove data
- Sorting and searching
- Simple numeric operations



# Common Tasks have Common Goals

## Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

- We can (already) write programs that are very similar independent of the data type used (Recall templates)
  - Using an `int` is not that different from using a `double`
  - Using a `vector<int>` is not that different from using a `vector<string>`
- We would like to write common programming tasks so that we do not have to re-do the work each time we find a new way of storing the data or a slightly different way of interpreting the data
  - Finding a value in a `vector` is not all that different from finding a value in a `list` or an array
  - Looking for a `string` ignoring case is not all that different from looking at a `string` not ignoring case
  - Graphing experimental data with exact values is not all that different from graphing data with rounded values
  - Copying a file is not all that different from copying a `vector`



# Ideals for Commonly used Common Codes

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

- Code that is
  - Easy to read
  - Easy to modify
  - Regular
  - Short
  - Fast
- Uniform access to data
  - Independently of how it is stored
  - Independently of its type
- Type-safe access to data
- Easy traversal of data
- Compact storage of data
- Fast
  - Retrieval of data
  - Addition of data
  - Deletion of data
- Standard versions of the most common algorithms
  - Copy, find, search, sort, sum, ...



# Examples

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

- Sort a vector of strings
- Find a number in a phone book, given a name
- Find the highest temperature
- Find all values larger than 800
- Find the first occurrence of the value 17
- Sort the telemetry records by unit number
- Sort the telemetry records by time stamp
- Find the first value larger than “Petersen”?
- What is the largest amount seen?
- Find the first difference between two sequences
- Compute the pairwise product of the elements of two sequences
- What are the highest temperatures for each day in a month?
- What are the top 10 best-sellers?
- What is the entry for “C++” (say, in Google)?
- What is the sum of the elements?



# Generic Programming

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

- Generalize algorithms
  - Sometimes called *lifting an algorithm*
- The aim (for the end user) is
  - Increased correctness
    - ▷ Through better specification
  - Greater range of uses
    - ▷ Possibilities for re-use
  - Better performance
    - ▷ Through wider use of tuned libraries
    - ▷ Unnecessarily slow code will eventually be thrown away
- Go from the concrete to the more abstract
  - The other way most often leads to bloat





# Lifting example: Concrete Algorithms

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

```
// Sum in Array: one concrete algorithm (doubles in array)
```

```
double sum(double array[], int n) { // data
    double s = 0;
    for (int i = 0;
        i < n;          // not at end
        ++i)            // get next data element
        s = s + array[i]; // get value
    return s;
}
```

```
// Sum in List: another concrete algorithm (ints in list)
```

```
struct Node { Node* next; int data; };
int sum(Node* first) { // data
    int s = 0;
    while (first) {      // not at end : terminates on null pointer
        s = s + first->data; // get value
        first = first->next; // get next data element
    }
    return s;
}
```



# Lifting example: Abstract the data structure

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

// pseudo-code for a more general version of both algorithms

```
int sum(data) {                                // somehow parameterize with the data structure
    int s = 0;                                  // initialize
    while (not at end) {                        // loop through all elements
        s = s + get value;                      // compute sum
        get next data element;
    }
    return s;                                  // return result
}
```

- We need three operations (on the data structure):

- not at end
- get value
- get next data element



# Lifting example: Using template

## Module M43

Partha Pratim Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

// Concrete STL-style code for a more general version of both algorithms

```
template<class Iter, class T>           // Iter should be an Input_iterator
                                         // T should be something we can + and =
T sum(Iter first, Iter last, T s) {    // T is the accumulator type
    while (first != last) {            // not at end
        s = s + *first;                // get value
        ++first;                       // get next data element
    }
    return s;
}
```

- Let the user initialize the accumulator

```
float a[] = { 1,2,3,4,5,6,7,8 }; // a[0], a[0], ..., a[7]
double d = 0;
d = sum(a, a+sizeof(a)/sizeof(*a), d); // [&a[0],&a[8])= {a[0], a[0], ..., a[7]}
```



# Lifting example

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

- Almost the standard library accumulate
  - A bit for terseness is simplified
- Works for
  - arrays
  - `vectors`
  - `lists`
  - `istreams`
  - . . .
- Runs as fast as *hand-crafted* code
  - Given decent inlining
- The code's requirements on its data has become explicit
  - We understand the code better



# Basic Model: Algorithms ==> Iterators <== Containers

## Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

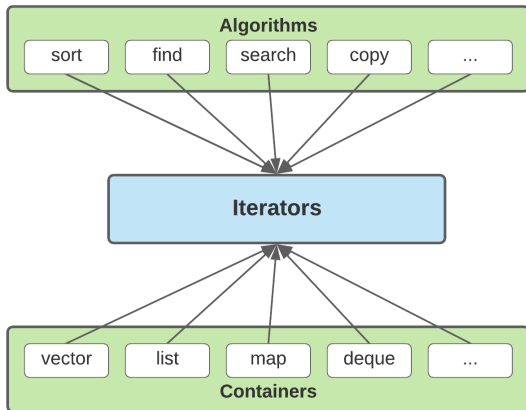
Common Tasks

Lifting Example

Model

Examples

Module Summary



## • Separation of Concerns

- *Algorithms* manipulate data, but do not know about *Containers*
- *Containers* store data, but do not know about *Algorithms*
- *Algorithms* and *Containers* interact through *Iterators*
- Each *Container* has its *own iterator types*



# Basic Model: Iterators

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

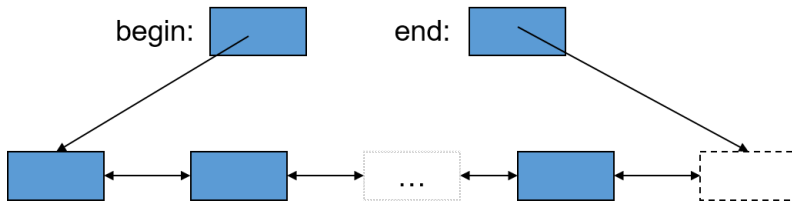
Lifting Example

Model

Examples

Module Summary

- A pair of iterators defines a sequence
  - The *beginning* (points to the *first element* – if any)
  - The *end* (points to the *one-beyond-the-last element*)



- An iterator is a type that supports the *iterator operations*
  - `++` Go to next element
  - `*` Get value
  - `==` Does this iterator point to the same element as that iterator?
- Some iterators support more operations (for example, `--`, `+`, and `[ ]`)



# Basic Model: Algorithms + Iterators

## Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

Common Tasks

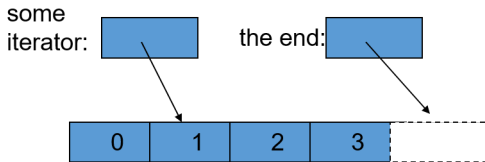
Lifting Example

Model

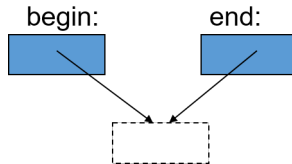
Examples

Module Summary

- An iterator points to (refers to, denotes) an element of a sequence
- The end of the sequence is *one past the last element*
  - not *the last element*
  - That is necessary to elegantly represent an empty sequence
  - One-past-the-last-element is not an element
    - ▷ You can compare an iterator pointing to it
    - ▷ You cannot dereference it (read its value)
- Returning the end of the sequence is the standard idiom for *not found* or *unsuccessful*



An empty sequence:





# Basic Model: Containers + Iterators

## Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

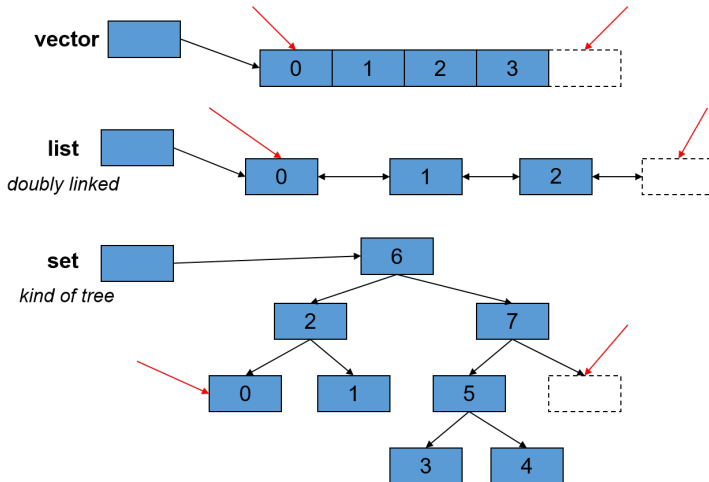
Common Tasks

Lifting Example

Model

Examples

Module Summary







# Algorithm: find()

Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

```
// Find the first element that equals a value
template<class In, class T>
In find(In first, In last, const T& val) {
    while (first != last && *first != val) ++first;
    return first;
}

void f(vector<int>& v, int x) { // works for vector of ints
    vector<int>::iterator p = find(v.begin(), v.end(), x);
    if (p != v.end()) /* we found x */
        // ...
}

void f(list<string>& v, string x) { // works for list of strings
    list<string>::iterator p = find(v.begin(), v.end(), x);
    if (p != v.end()) /* we found x */
        // ...
}

void f(set<double>& v, double x) { // works for set of doubles
    set<double>::iterator p = find(v.begin(), v.end(), x);
    if (p != v.end()) /* we found x */
        // ...
}
```



# Algorithm: find\_if()

Module M43

Partha Pratim Das

Objectives & Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic

Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

```
// Find the first element that matches a criterion (predicate)
template<class In, class Pred>
In find_if(In first, In last, Pred pred) {
    while (first != last && !pred(*first)) ++first;
    return first;
}

void f(vector<int>& v) {
    vector<int>::iterator p = find_if(v.begin(), v.end, Odd()); // Here, a predicate takes
                                                                // one argument and returns a bool
    if (p != v.end()) { /* we found an odd number */ }
    // ...
}
```

- A predicate (often of one argument) is a function or a function object returns a **bool** given the argument/s. For example

```
// A function
bool odd(int i) { return i % 2; } // % is the remainder (modulo) operator
odd(7);                          // call odd: is 7 odd?
```

```
// A function object (Module 40)
struct Odd { bool operator()(int i) const { return i % 2; } };
Odd odd;    // make an object odd of type Odd
odd(7);     // call odd: is 7 odd?
```



# Module Summary

## Module M43

Partha Pratim  
Das

Objectives &  
Outlines

Standard Library

C Std. Lib.

C++ Std. Lib.

std

Header Conventions

Generic  
Programming

Common Tasks

Lifting Example

Model

Examples

Module Summary

- Overview of Standard Library components of C++
- Learnt fundamentals of generic programming



## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

# Programming in Modern C++

## Module M44: C++ Standard Library (STL): Part 2

Partha Pratim Das

Department of Computer Science and Engineering  
Indian Institute of Technology, Kharagpur

*ppd@cse.iitkgp.ac.in*

*All url's in this module have been accessed in September, 2021 and found to be functional*



# Module Recap

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- Overview of Standard Library components of C++
- Learnt fundamentals of generic programming



# Module Objectives

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- To understand Standard Template Library (STL)
- To understand common containers (data structure) and their use



# Module Outline

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

`vector`

`list`

`map`

`set`

Module Summary

- 1 The STL
  - Policy Parameterization
- 2 Common Standard Library Components
  - `vector`
  - `list`
  - `map`
  - `set`
- 3 Module Summary



# The STL

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

### The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

# The STL

## Source:

- [Chapter 20 The STL \(containers, iterators, and algorithms\)](#), Bjarne Stroustrup
- [Chapter 21 The STL \(maps and algorithms\)](#), Bjarne Stroustrup





# STL: Standard Template Library

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- Part of the ISO C++ Standard Library
- Has four components:
  - `containers`
  - `iterators`
  - `algorithms`
  - `functions`
- Mostly non-numerical
  - Only 4 standard algorithms specifically do computation
    - ▷ `accumulate`, `inner_product`, `partial_sum`, `adjacent_difference`
- Handles textual data as well as numeric data
  - For example, `string`
- Deals with organization of code and data
  - Built-in types, user-defined types, and data structures
- Optimizing disk access is among its original uses
  - Performance is always a key concern



# The STL

## Module M44

Partha Pratim Das

Objectives & Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- Designed by Alex Stepanov
- General aim: *Most general, most efficient, most flexible* representation of concepts (ideas, algorithms)
  - Represent separate concepts separately in code
  - Combine concepts freely wherever meaningful
- General aim to make programming *like math*
  - or even *Good programming is math*
  - works for integers, for floating-point numbers, for polynomials, for ...





# The STL

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- An ISO C++ standard framework of about 10 containers and about 60 algorithms connected by iterators
  - Other organizations provide more containers and algorithms in the style of the STL
    - ▷ [Boost.org](https://boost.org): Boost provides free peer-reviewed portable C++ source libraries. It has several [containers](#) include a number of [non-standard containers](#) (like [stable\\_vector](#), [flat\\_\(multi\)map/set](#) associative containers, [slist](#), [static\\_vector](#), and [small\\_vector](#))
    - ▷ [MSVC STL](#): Microsoft VC++ Standard Library has been released as open source: [Open Sourcing MSVC's STL](#), 2019
    - ▷ [Dinkumware Standard C++ Library](#)
    - ▷ [SGI](#)
- The best known and most widely used example of generic programming



# Basic Model: Algorithms ==> Iterators <== Containers: Recap (Module 43)

## Module M44

Partha Pratim Das

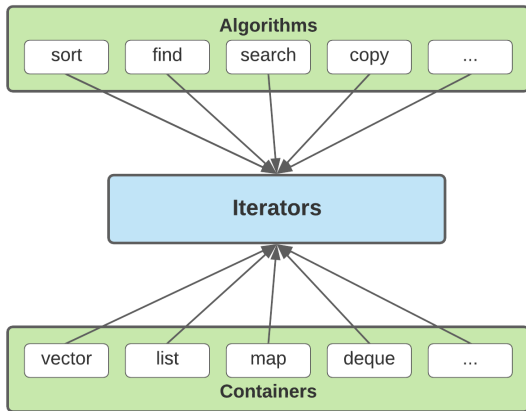
Objectives & Outlines

The STL

Policy  
Parameterization

Common  
Components  
vector  
list  
map  
set

Module Summary



## • Separation of Concerns

- *Algorithms* manipulate data, but do not know about *Containers*
- *Containers* store data, but do not know about *Algorithms*
- *Algorithms* and *Containers* interact through *Iterators*
- Each *Container* has its *own iterator types*



# Basic Model: Iterators: Recap (Module 43)

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

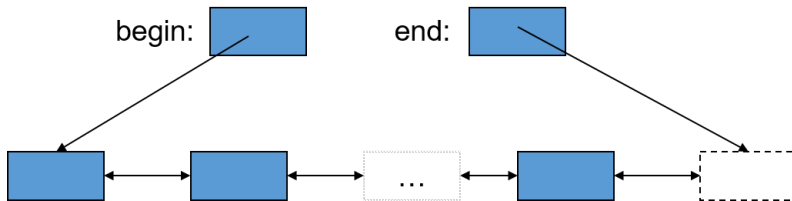
Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- A pair of iterators defines a sequence
  - The *beginning* (points to the *first element* – if any)
  - The *end* (points to the *one-beyond-the-last element*)



- An iterator is a type that supports the *iterator operations*
  - `++` Go to next element
  - `*` Get value
  - `==` Does this iterator point to the same element as that iterator?
- Some iterators support more operations (for example, `--`, `+`, and `[ ]`)



# Basic Model: Algorithms + Iterators: Recap (Module 43)

## Module M44

Partha Pratim Das

Objectives & Outlines

The STL

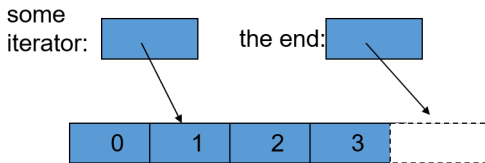
Policy  
Parameterization

Common  
Components

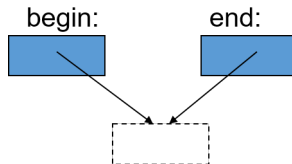
vector  
list  
map  
set

Module Summary

- An iterator points to (refers to, denotes) an element of a sequence
- The end of the sequence is *one past the last element*
  - not *the last element*
  - That is necessary to elegantly represent an empty sequence
  - One-past-the-last-element is not an element
    - ▷ You can compare an iterator pointing to it
    - ▷ You cannot dereference it (read its value)
- Returning the end of the sequence is the standard idiom for *not found* or *unsuccessful*



An empty sequence:





# Basic Model: Containers + Iterators: Recap (Module 43)

Module M44

Partha Pratim Das

Objectives & Outlines

The STL

Policy  
Parameterization

Common  
Components

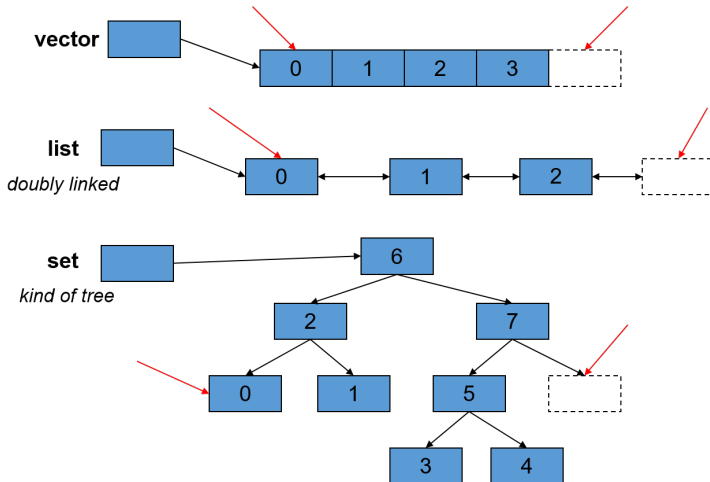
vector

list

map

set

Module Summary





# Algorithm: find(): Recap (Module 43)

Module M44

Partha Pratim Das

Objectives & Outlines

The STL

Policy  
Parameterization

Common Components

vector  
list  
map  
set

Module Summary

```
// Find the first element that equals a value
template<class In, class T>
In find(In first, In last, const T& val) {
    while (first != last && *first != val) ++first;
    return first;
}

void f(vector<int>& v, int x) { // works for vector of ints
    vector<int>::iterator p = find(v.begin(), v.end(), x);
    if (p != v.end()) /* we found x */
        // ...
}

void f(list<string>& v, string x) { // works for list of strings
    list<string>::iterator p = find(v.begin(), v.end(), x);
    if (p != v.end()) /* we found x */
        // ...
}

void f(set<double>& v, double x) { // works for set of doubles
    set<double>::iterator p = find(v.begin(), v.end(), x);
    if (p != v.end()) /* we found x */
        // ...
}
```





# Algorithm: find\_if(): Recap (Module 43)

Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

```
// Find the first element that matches a criterion (predicate)
template<class In, class Pred>
In find_if(In first, In last, Pred pred) {
    while (first != last && !pred(*first)) ++first;
    return first;
}

void f(vector<int>& v) {
    vector<int>::iterator p = find_if(v.begin(), v.end, Odd()); // Here, a predicate takes
                                                                // one argument and returns a bool
    if (p != v.end()) { /* we found an odd number */ }
    // ...
}
```

- A predicate (often of one argument) is a function or a function object returns a **bool** given the argument/s. For example

```
// A function
bool odd(int i) { return i % 2; } // % is the remainder (modulo) operator
odd(7);                          // call odd: is 7 odd?
```

```
// A function object (Module 40)
struct Odd { bool operator()(int i) const { return i % 2; } };
Odd odd;    // make an object odd of type Odd
odd(7);     // call odd: is 7 odd?
```



# Policy Parameterization

- When we have a useful algorithm, we may want to *parameterize* it by a *policy*
  - For example, we need to *parameterize sort* by the *comparison criteria*

```
struct Record {
    string name;    // standard string for ease of use
    char addr[24]; // old C-style string to match database layout
    // ...
};
vector<Record> vr;
// ...
sort(vr.begin(), vr.end(), Cmp_by_name()); // sort by name
sort(vr.begin(), vr.end(), Cmp_by_addr()); // sort by addr

// Different comparisons for Rec objects
struct Cmp_by_name {
    bool operator()(const Record& a, const Record& b) const
    { return a.name < b.name; } // look at the name field of Record
};
struct Cmp_by_addr {
    bool operator()(const Record& a, const Record& b) const
    { return 0 < strncmp(a.addr, b.addr, 24); } // look at the addr field of Record
};
// Note how the comparison function objects are used to hide ugly and error-prone code
```



# Policy Parameterization

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector

list

map

set

Module Summary

- *Lambda* or *closure object* may be used to parameterize a *policy*

```
vector<Record> vr;  
// ...  
sort(vr.begin(), vr.end(),  
    [] (const Record& a, const Record& b) // lambda expression as policy [C++11]  
    { return a.name < b.name; }          // sort by name  
    );  
  
sort(vr.begin(), vr.end(),  
    [] (const Record& a, const Record& b) // lambda expression as policy [C++11]  
    { return 0 < strncmp(a.addr, b.addr, 24); } // sort by addr  
    );  
  
// A lambda expression is an anonymous function - a function without a name
```



# Policy Parameterization

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- Use a *named object* as argument
  - If you want to do something complicated
  - If you feel the need for a comment
  - If you want to do the same in several places
- Use a *lambda expression* as argument [C++11]
  - If what you want is short and obvious
- Choose based on *clarity of code*
  - There are no performance differences between function objects and lambdas
  - Function objects (and lambdas) tend to be faster than function arguments



# Common Standard Library Components

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

`vector`

`list`

`map`

`set`

Module Summary

## Common Standard Library Components



# Common Standard Library Headers

Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- `<iostream>` // I/O streams, `cout`, `cin`, ... (Module 42)
- `<fstream>` // file streams (Module 42)
- Containers
  - `<string>` // many Modules
  - `<vector>` // many Modules
  - `<map>`
  - `<list>`
  - `<set>`
  - `<unordered_map>` // hash table [C++11]
  - ...
- `<algorithm>` // sort, copy, ...
- `<numeric>` // accumulate, inner\_product, ...
- `<functional>` // function objects (Module 40)

**More components will be covered in weeks 10-12 in the course of discussions on [C++11] onward**



# vector

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

**vector**

list

map

set

Module Summary

```
template<class T> class vector {  
    T* elements;  
    // ...  
    using value_type = T;  
    using iterator = ???; // the type of an iterator is implementation defined  
                           // and it (usefully) varies (for example, range checked iterators)  
                           // a vector iterator could be a pointer to an element  
    using const_iterator = ???;  
  
    iterator begin();           // points to first element  
    const_iterator begin() const;  
  
    iterator end();           // points to one beyond the last element  
    const_iterator end() const;  
  
    iterator insert(iterator p, const T& v); // insert a new element v before p  
  
    iterator erase(iterator p);           // remove element pointed to by p  
};
```



# insert() into vector

Module M44

Partha Pratim Das

Objectives & Outlines

The STL

Policy  
Parameterization

Common  
Components

**vector**

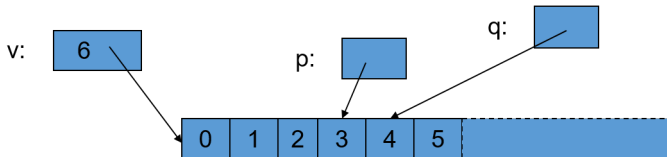
list

map

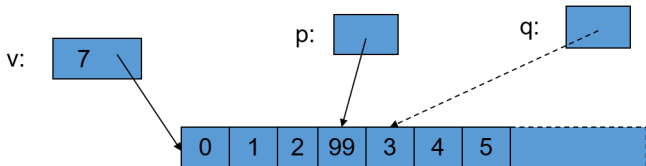
set

Module Summary

```
vector<int>::iterator p = v.begin(); ++p; ++p; ++p;  
vector<int>::iterator q = p; ++q;
```



```
p = v.insert(p, 99); // leaves p pointing at the inserted element
```



```
// Note: q is invalid after the insert()
```

```
// Note: Some elements moved; all elements could have moved
```





# erase() from vector

Module M44

Partha Pratim Das

Objectives & Outlines

The STL

Policy  
Parameterization

Common  
Components

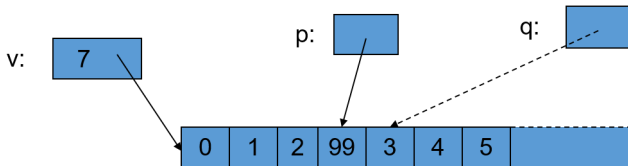
**vector**

list

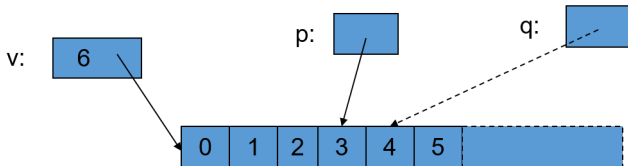
map

set

Module Summary



```
p = v.erase(p); // leaves p pointing at the element after the erased one
```



```
// Note: vector elements move when we insert() or erase()
```

```
// Note: Iterators into a vector are invalidated by insert() and erase()
```



# Ways of traversing a vector

Module M44

Partha Pratim Das

Objectives & Outlines

The STL

Policy  
Parameterization

Common  
Components

**vector**  
list  
map  
set

Module Summary

- ```
for(int i = 0; i < v.size(); ++i) // index style. why int?  
    ... // do something with v[i]  
  
for(vector<T>::size_type i = 0; i < v.size(); ++i) // index style, always correct  
    ... // do something with v[i]  
  
for(vector<T>::iterator p = v.begin(); p != v.end(); ++p) // iterator style  
    ... // do something with *p
```
- Know both ways (iterator and subscript)
    - The subscript style is used in essentially every language
    - The iterator style is used in C (pointers only) and C++
    - The iterator style is used for standard library algorithms
    - The subscript style does not work for lists (in C++ and in most languages)
  - Use either way for vectors
    - There are no fundamental advantages of one style over the other
    - But the iterator style works for all sequences
    - Prefer `size_type` over plain `int`
      - ▷ pedantic, but quiets compiler and prevents rare errors



# Ways of traversing a vector

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

**vector**  
list  
map  
set

Module Summary

```
for(vector<T>::iterator p = v.begin(); p!=v.end(); ++p) // iterator style  
    ... // do something with *p
```

```
for(vector<T>::value_type x : v) // iterator style, range for [C++11]  
    ... // do something with x
```

```
for(auto& x : v) // iterator style, range for [C++11]  
    ... // do something with x
```

- Range for [C++11]
  - Use for the simplest loops
    - ▷ Every element from `begin()` to `end()`
  - Over one sequence
  - When we do not need to look at more than one element at a time
  - When we do not need to know the position of an element



# list: Doubly Linked List

Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector

list

map

set

Module Summary

```
template<class T> struct Link {
    T value;
    Link* pre;
    Link* post;
}
template<class T> class list {
    Link* elements;
    // ...
    using value_type = T;
    using iterator = ???; // the type of an iterator is implementation defined
                          // and it (usefully) varies (for example, range checked iterators)
                          // a vector iterator could be a pointer to a link node
    using const_iterator = ???;

    iterator begin();           // points to first element
    const_iterator begin() const;

    iterator end();            // points to one beyond the last element
    const_iterator end() const;

    iterator insert(iterator p, const T& v); // insert a new element v before p

    iterator erase(iterator p);           // remove element pointed to by p
};
```



# insert() into list

Module M44

Partha Pratim Das

Objectives & Outlines

The STL

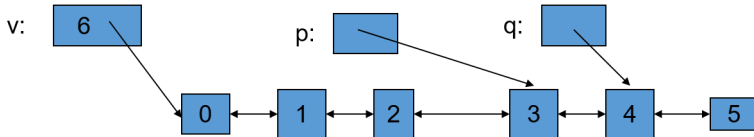
Policy  
Parameterization

Common Components

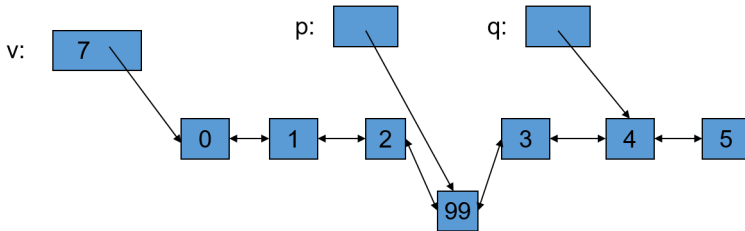
vector  
list  
map  
set

Module Summary

```
list<int>::iterator p = v.begin(); ++p; ++p; ++p;  
list<int>::iterator q = p; ++q;
```



```
p = v.insert(p, 99); // leaves p pointing at the inserted element
```



```
// Note: q is unaffected
```

```
// Note: No elements moved around
```

Programming in Modern C++

Partha Pratim Das

M44.26



# erase() from list

Module M44

Partha Pratim Das

Objectives & Outlines

The STL

Policy  
Parameterization

Common  
Components

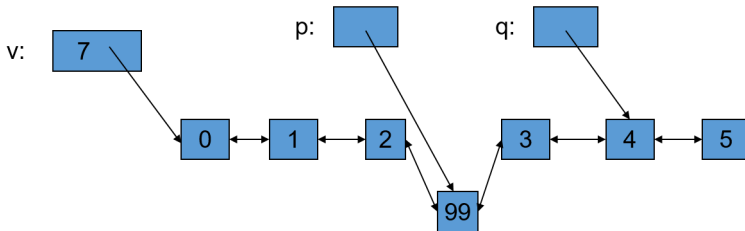
vector

list

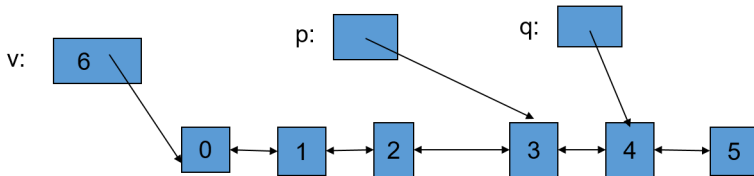
map

set

Module Summary



```
p = v.erase(p); // leaves p pointing at the element after the erased one
```



```
// Note: list elements do not move when we insert() or erase()
```



# vector vs. list

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector

list

map

set

Module Summary

- By default, use a **vector**
  - We need a reason not to
  - We can *grow* a vector (for example, using `push_back()`)
  - We can `insert()` and `erase()` in a **vector**
  - Vector elements are *compactly stored* and *contiguous*
  - For small vectors of small elements all operations are fast
    - ▷ compared to lists
- If we do not want elements to move, use a list
  - We can *grow* a list (for example, using `push_back()` and `push_front()`)
  - We can `insert()` and `erase()` in a **list**
  - List elements are *separately allocated*
- Note that there are more containers like
  - `map`
  - `unordered_map` [C++11]



# map: An associative array

Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- For a `vector`, we subscript using an integer
- For a `map`, we can define the subscript to be (just about) any type
- After `vector`, `map` is the most useful standard library container
  - Maps (and/or hash tables) are the backbone of scripting languages
- A `map` is really an *ordered balanced binary tree*, by default ordered by `<` (less than)

// note the similarity to `vector` and `list`

```
template<class Key, class Value> class map {
```

```
    // ...
```

```
    using value_type = pair<Key, Value>; // a map deals in (Key, Value) pairs
```

```
    using iterator = ???; // Some implementation defined type - probably a pointer to a tree node
```

```
    using const_iterator = ???;
```

```
    iterator begin(); // points to first element
```

```
    iterator end();   // points to one beyond the last element
```

```
    Value& operator[](const Key&); // get Value for Key; creates pair if necessary, using Value()
```

```
    iterator find(const Key& k);   // is there an entry for k?
```

```
    pair<iterator, bool> insert(const value_type&); // insert new (Key, Value) pair
```

```
                                // the bool is false if insert failed
```

```
    void erase(iterator p);        // remove element pointed to by p
```

```
};
```

Programming in Modern C++

Partha Pratim Das

M44.29





# map: Example: Simple Use

Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector

list

map

set

Module Summary

```
#include <iostream>
#include <map>
using namespace std;

int main() { map<char,int> myMap; // Key = char, Value = int

    myMap['a'] = 10;                // initializing a map. using index
    myMap.insert(pair<char, int>('c', 30)); // using insert. myMap['c'] = 30;
    map<char, int>::iterator it = myMap.begin();
    myMap.insert(it, pair<char, int>('b', 20)); // using insert with hint. myMap['b'] = 20
    myMap['d'] = 40;

    for(it = myMap.begin(); it != myMap.end(); ++it) // print myMap
        cout << it->first << " => " << it->second << endl;

    it = myMap.find('c'); // search myMap for key = 'c'
    if (it != myMap.end())
        cout << "Value of myMap['c'] = " << it->second << endl;
}

a => 10
b => 20
c => 30
d => 40
Value of myMap['c'] = 30
```



# map: Example: Counting Words

Module M44

Partha Pratim Das

Objectives & Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

```
#include <iostream>
#include <map>
#include <string>
using namespace std;
int main() { map<string, int> words; // keep (word, frequency) pairs. Key type = string, Value type = int
    for (string s; cin >> s;)
        ++words[s]; // words is indexed by string, words[s] returns int&, the int values are set to 0
    for (const auto& p: words) // Iterating the map in [C++11] style
        cout << p.first << ": " << p.second << "\n";
}
```

## Input: words

Twinkle, twinkle, little star  
How I wonder what you are  
Up above the world so high  
Like a diamond in the sky  
Twinkle, twinkle, little star  
How I wonder what you are

Twinkle, twinkle, little star  
How I wonder what you are  
Up above the world so high  
Like a diamond in the sky  
Twinkle, twinkle, little star  
How I wonder what you are

## Output: : frequency

|             |            |           |             |
|-------------|------------|-----------|-------------|
| How: 4      | above: 2   | little: 4 | twinkle,: 4 |
| I: 4        | are: 4     | sky: 2    | what: 4     |
| Like: 2     | diamond: 2 | so: 2     | wonder: 4   |
| Twinkle,: 4 | high: 2    | star: 4   | world: 2    |
| Up: 2       | in: 2      | the: 4    | you: 4      |
| a: 2        |            |           |             |



# set

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

vector  
list  
map  
set

Module Summary

- **sets** are containers that store *unique elements* following a specific order
- In a set, the value of an element also identifies it (the value is itself the key, of type T), and each value must be unique
- Internally, the elements in a set are always sorted following a specific strict weak ordering criterion indicated by its internal comparison object (of type Compare)
- set containers are generally slower than unordered\_set containers
- Sets are typically implemented as *binary search trees*

```
template<class T> class set {  
    // ...  
    using value_type = T;  
  
    using iterator = ???; // Some implementation defined type - probably a pointer to a tree node  
    using const_iterator = ???;  
  
    iterator begin(); // points to first element  
    iterator end();   // points to one beyond the last element  
  
    pair<iterator, bool> insert(const value_type&); // insert new (Key, Value) pair  
                                                    // the bool is false if insert failed  
    iterator erase(const_iterator p);             // remove element pointed to by p  
};
```



# Module Summary

## Module M44

Partha Pratim  
Das

Objectives &  
Outlines

The STL

Policy  
Parameterization

Common  
Components

`vector`

`list`

`map`

`set`

Module Summary

- Learnt Standard Template Library (STL) with common components
- Learnt useful containers and their use



## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm  
copy

numeric  
accumulate  
inner\_product

functional

Module Summary

# Programming in Modern C++

## Module M45: C++ Standard Library (STL): Part 3

Partha Pratim Das

Department of Computer Science and Engineering  
Indian Institute of Technology, Kharagpur

*ppd@cse.iitkgp.ac.in*

*All url's in this module have been accessed in September, 2021 and found to be functional*



# Module Recap

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

- Learnt Standard Template Library (STL) with common components
- Learnt useful containers and their use



# Module Objectives

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

- Summarize containers in STL
- To take a look at a few important library components



# Module Outline

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm  
copy

numeric  
accumulate  
inner\_product

functional

Module Summary

- 1 Data Structures / Containers in C++
  - Containers in C++
- 2 algorithm Component
  - copy
- 3 numeric Component
  - accumulate
  - inner\_product
- 4 functional Component
- 5 Module Summary





# Data Structures / Containers in C++

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

# Data Structures / Containers in C++

## Source:

- [Container Classes](#), isocpp
- [Containers library](#), cppreference
- [Standard C++ Library reference: Containers](#), cplusplus



# Data Structures / Containers in C++

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm  
copy

numeric  
accumulate  
inner\_product

functional

Module Summary

- Like Stack, several other data structures are available in C++ standard library
- They are *ready-made* and *work like a data type*
- *Varied types of elements* can be used for C++ data structures
- **Data Structures** in C++ are commonly called **Containers**:
  - A container is a *holder object* that stores a *collection of other objects* (its elements)
  - They are implemented as *class templates* allowing great flexibility in the types supported as elements
  - The container
    - ▷ *manages the storage space* for its elements
    - ▷ provides member *functions to access* them
    - ▷ supports *iterators* - reference objects with similar properties to pointers
  - Many containers have several *member functions in common*, and *share functionalities* - easy to learn and remember



# Data Structures / Containers in C++

## Module M45

Partha Pratim Das

Objectives & Outlines

Data Structures / Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

- **Data Structures** in C++ are commonly called **Containers**:
  - `vector`, `list` and `deque` are **Sequence Containers**
  - `map`, `set`, `multimap`, and `multiset` are **Associative Containers**. Also, `unordered_map` (hash table) and `unordered_set` in C++11
  - `array` (language feature), `string`, `stack`, `queue`, `priority_queue`, and `bitset` are **almost Containers**. C++11 has `array` too
  - `stack`, `queue` and `priority_queue` are implemented as **Container Adaptors**
    - ▷ Container adaptors are *not full container classes*, but classes that provide a *specific interface relying on an object of one of the container classes* (such as `deque` or `list`) to handle the elements
    - ▷ The underlying container is encapsulated in such a way that its elements are accessed by the members of the container adaptor independently of the underlying container class used



# Data Structures / Containers in C++

Module M45

Partha Pratim Das

Objectives & Outlines

Data Structures / Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

| Container                                                                                                                                                                                                                                  | Class Template     | Remarks                                                                                    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|--------------------------------------------------------------------------------------------|
| <b>Sequence containers:</b> <i>Elements are ordered in a strict sequence and are accessed by their position in the sequence</i>                                                                                                            |                    |                                                                                            |
| <code>array</code> (C++11)                                                                                                                                                                                                                 | Array class        | 1D array of <i>fixed-size</i>                                                              |
| <code>vector</code>                                                                                                                                                                                                                        | Vector             | 1D array of <i>fixed-size</i> that can <i>change in size</i>                               |
| <code>deque</code>                                                                                                                                                                                                                         | Double ended queue | <i>Dynamically sized</i> , can be expanded / contracted on <i>both ends</i>                |
| <code>forward_list</code> (C++11)                                                                                                                                                                                                          | Forward list       | <i>Const. time insert / erase</i> anywhere, done as <i>singly-linked lists</i>             |
| <code>list</code>                                                                                                                                                                                                                          | List               | <i>Const. time insert / erase</i> anywhere, iteration in <i>both directions</i>            |
| <b>Container adaptors:</b> <i>Sequence containers adapted with specific protocols of access like LIFO, FIFO, Priority</i>                                                                                                                  |                    |                                                                                            |
| <code>stack</code>                                                                                                                                                                                                                         | LIFO stack         | Underlying container is <code>deque</code> (default) or as specified                       |
| <code>queue</code>                                                                                                                                                                                                                         | FIFO queue         | Underlying container is <code>deque</code> (default) or as specified                       |
| <code>priority_queue</code>                                                                                                                                                                                                                | Priority queue     | Underlying container is <code>vector</code> (default) or as specified                      |
| <b>Associative containers:</b> <i>Elements are referenced by their key and not by their absolute position in the container</i><br><i>They are typically implemented as binary search trees and needs the elements to be comparable</i>     |                    |                                                                                            |
| <code>set</code>                                                                                                                                                                                                                           | Set                | Stores <i>unique elements</i> in a <i>specific order</i>                                   |
| <code>multiset</code>                                                                                                                                                                                                                      | Multiple-key set   | Stores elements in <i>an order</i> with <i>multiple equivalent values</i>                  |
| <code>map</code>                                                                                                                                                                                                                           | Map                | Stores <i>&lt;key, value&gt;</i> in <i>an order</i> with <i>unique keys</i>                |
| <code>multimap</code>                                                                                                                                                                                                                      | Multiple-key map   | Stores <i>&lt;key, value&gt;</i> in <i>an order</i> with <i>multiple equivalent values</i> |
| <b>Unordered associative containers:</b> <i>Elements are referenced by their key and not by their absolute position in the container</i><br><i>Implemented using a hash table of keys and has fast retrieval of elements based on keys</i> |                    |                                                                                            |
| <code>unordered_set</code> (C++11)                                                                                                                                                                                                         | Unordered Set      | Stores <i>unique elements</i> in <i>no particular order</i>                                |
| <code>unordered_multiset</code> (C++11)                                                                                                                                                                                                    | Unordered Multiset | Stores elements in <i>no order</i> with <i>multiple equivalent values</i>                  |
| <code>unordered_map</code> (C++11)                                                                                                                                                                                                         | Unordered Map      | Stores <i>&lt;key, value&gt;</i> in <i>no order</i> with <i>unique keys</i>                |
| <code>unordered_multimap</code> (C++11)                                                                                                                                                                                                    | Unordered Multimap | Stores <i>&lt;key, value&gt;</i> in <i>no order</i> with <i>multiple equivalent values</i> |



# STL Containers

Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

```
template < class T, class Alloc = allocator<T> > class vector; // generic template
template < class T, class Alloc = allocator<T> > class deque;
template < class T, class Alloc = allocator<T> > class list;
template < class T,                                // set::key_type/value_type
    class Compare = less<T>,                        // set::key_compare/value_compare
    class Alloc = allocator<T>                    // set::allocator_type
    > class set;
template < class T,                                // multiset::key_type/value_type
    class Compare = less<T>,                        // multiset::key_compare/value_compare
    class Alloc = allocator<T> >                // multiset::allocator_type
    > class multiset;
template < class Key,                                // map::key_type
    class T,                                        // map::mapped_type
    class Compare = less<Key>,                    // map::key_compare
    class Alloc = allocator<pair<const Key,T> >    // map::allocator_type
    > class map;
template < class Key,                                // multimap::key_type
    class T,                                        // multimap::mapped_type
    class Compare = less<Key>,                    // multimap::key_compare
    class Alloc = allocator<pair<const Key,T> >    // multimap::allocator_type
    > class multimap;
template <class T, class Container = deque<T> > class stack;
template <class T, class Container = deque<T> > class queue;
template <class T, class Container = vector<T>,
    class Compare = less<typename Container::value_type> > class priority_queue;
```



# STL Containers

## Module M45

Partha Pratim Das

Objectives & Outlines

Data Structures / Containers

Containers in C++

algorithm  
copy

numeric  
accumulate  
inner\_product

functional

Module Summary

| Member Type                   | Definition                                                                                             | Notes                                                         |
|-------------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <i>value_type</i>             | Template parameter <b>T</b>                                                                            |                                                               |
| <i>allocator_type</i>         | Template parameter <b>Alloc</b>                                                                        | defaults to: <code>allocator&lt;value_type&gt;</code>         |
| <i>reference</i>              | <code>allocator_type::reference</code>                                                                 | for the default allocator: <code>value_type&amp;</code>       |
| <i>const_reference</i>        | <code>allocator_type::const_reference</code>                                                           | for the default allocator: <code>const value_type&amp;</code> |
| <i>pointer</i>                | <code>allocator_type::pointer</code>                                                                   | for the default allocator: <code>value_type*</code>           |
| <i>const_pointer</i>          | <code>allocator_type::const_pointer</code>                                                             | for the default allocator: <code>const value_type*</code>     |
| <i>iterator</i>               | a random access iterator to <code>value_type</code>                                                    | convertible to <code>const_iterator</code>                    |
| <i>const_iterator</i>         | a random access iterator to <code>const value_type</code>                                              |                                                               |
| <i>reverse_iterator</i>       | <code>reverse_iterator&lt;iterator&gt;</code>                                                          |                                                               |
| <i>const_reverse_iterator</i> | <code>reverse_iterator&lt;const_iterator&gt;</code>                                                    |                                                               |
| <i>difference_type</i>        | a signed integral type, identical to:<br><code>iterator_traits&lt;iterator&gt;::difference_type</code> | usually the same as <code>ptrdiff_t</code>                    |
| <i>size_type</i>              | an unsigned integral type that can represent any non-negative value of <code>difference_type</code>    | usually the same as <code>size_t</code>                       |
| <i>key_type</i>               | Template parameter <b>T</b>                                                                            |                                                               |
| <i>value_type</i>             | Template parameter <b>T</b>                                                                            |                                                               |
| <i>key_compare</i>            | Template parameter <b>Compare</b>                                                                      | defaults to: <code>less&lt;key_type&gt;</code>                |
| <i>value_compare</i>          | Template parameter <b>Compare</b>                                                                      | defaults to: <code>less&lt;value_type&gt;</code>              |



# Operations in STL Containers

## Module M45

Partha Pratim Das

Objectives & Outlines

Data Structures / Containers

Containers in C++

algorithm  
copy

numeric  
accumulate  
inner\_product

functional

Module Summary

| Container                                                                                                                                                                                                      | Capacity                       | Access                      | Modifier                                                    | Observers              | Operations                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|-----------------------------|-------------------------------------------------------------|------------------------|---------------------------------------------------------|
| vector                                                                                                                                                                                                         | resize capacity<br>reserve     | operator[] at<br>front back | assign push_back<br>pop_back                                |                        |                                                         |
| deque                                                                                                                                                                                                          | resize                         | operator[] at<br>front back | assign push_back<br>push_front pop_back<br>pop_front        |                        |                                                         |
| list                                                                                                                                                                                                           |                                | front back                  | assign push_back<br>push_front pop_back<br>pop_front resize |                        | splice remove remove_if<br>unique merge sort<br>reverse |
| set                                                                                                                                                                                                            |                                |                             |                                                             | key_comp<br>value_comp | find count lower_bound<br>upper_bound equal_range       |
| multiset                                                                                                                                                                                                       |                                |                             |                                                             | -do-                   | -do-                                                    |
| map                                                                                                                                                                                                            |                                | operator[]                  |                                                             | -do-                   | -do-                                                    |
| multimap                                                                                                                                                                                                       |                                |                             |                                                             | -do-                   | -do-                                                    |
| <b>Common:</b> (constructor) (destructor) operator= <b>Iterators:</b> begin end & rbegin rend <b>Capacity:</b> empty size max_size<br><b>Modifier:</b> insert erase swap clear <b>Allocator:</b> get_allocator |                                |                             |                                                             |                        |                                                         |
| stack                                                                                                                                                                                                          | empty size top push pop        |                             |                                                             |                        |                                                         |
| queue                                                                                                                                                                                                          | empty size front back push pop |                             |                                                             |                        |                                                         |
| priority_queue                                                                                                                                                                                                 | empty size top push pop        |                             |                                                             |                        |                                                         |



# algorithm Component

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

**algorithm**

copy

numeric

accumulate

inner\_product

functional

Module Summary

## algorithm Component

### Source:

- [Algorithms library](#), cppreference
- `<algorithm>`, cplusplus





# algorithm Component

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

**algorithm**

copy

numeric

accumulate

inner\_product

functional

Module Summary

- The header `<algorithm>` defines a collection of functions especially designed to be used on ranges of elements
- It provides STL-style algorithms
  - Takes one or more sequences
    - ▷ Usually as pairs of iterators
  - Takes one or more operations
    - ▷ Usually as function objects
    - ▷ Ordinary functions also work
  - Usually reports “failure” by returning the end of a sequence



# algorithm: Useful algorithms

## Module M45

Partha Pratim Das

Objectives & Outlines

Data Structures / Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

```
r = find(b,e,v)
r = find_if(b,e,p)
x = count(b,e,v)
x = count_if(b,e,p)
sort(b,e)
sort(b,e,p)
copy(b,e,b2)

unique_copy(b,e,b2)

merge(b,e,b2,e2,r)

r = equal_range(b,e,v)

equal(b,e,b2)
```

`r` points to the first occurrence of `v` in `[b,e)`  
`r` points to the first element `x` in `[b,e)` for which `p(x)`  
`x` is the number of occurrences of `v` in `[b,e)`  
`x` is the number of elements in `[b,e)` for which `p(x)`  
sort `[b,e)` using `<`  
sort `[b,e)` using `p`  
copy `[b,e)` to `[b2,b2+(e-b))`  
there had better be enough space after `b2`  
copy `[b,e)` to `[b2,b2+(e-b))`  
but do not copy adjacent duplicates  
merge two sorted sequence `[b2,e2)` and `[b,e)`  
into `[r,r+(e-b)+(e2-b2))`  
`r` is the subsequence of `[b,e)` with the value `v`  
(basically a binary search for `v`)  
do all elements of `[b,e)` and `[b2,b2+(e-b))` compare equal?

```
// b=> .begin() (start iterator), e=> .end() (end iterator), v=> value, p=> p(x) (predicate)
```



# copy

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate  
inner\_product

functional

Module Summary

The `copy` is available in `<algorithm>` and it

- Copies the elements in the range `[first, last)` into the range beginning at `result`
- Returns an iterator to the end of the destination range (which points to the element following the last element copied)
- The ranges shall not overlap in such a way that `result` points to an element in the range `[first, last)`

```
template<class InputIterator, class OutputIterator>
    OutputIterator copy(InputIterator first, InputIterator last, OutputIterator result) {
    while (first!=last) {
        *result = *first; // *res++ = *first++;
        ++result; ++first;
    }
    return result;
}
```



# copy: Copy from list to vector

Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

```
#include <iostream>
#include <vector>
#include <list>
#include <algorithm> // copy
using namespace std;

void f(vector<double>& vd, list<int>& li) {
    if (vd.size() < li.size()) { cerr << "target container too small" << endl; return; }

    cout << "dst before copy: "; for(auto& x : vd) cout << x << ' '; cout << endl;

    // note: different container types and different element types
    copy(li.begin(), li.end(), vd.begin());
    cout << "dst after copy: "; for(auto& x : vd) cout << x << ' '; cout << endl;

    sort(vd.begin(), vd.end());
    cout << "dst after sort: "; for(auto& x : vd) cout << x << ' '; cout << endl;
}

int main() {
    list<int> li = { 2, 7, 5, 6, 8, 9 }; // source container
    vector<double> vd(li.size());      // destination container

    cout << "src before copy: "; for(auto& x : li) cout << x << ' '; cout << endl;
    f(vd, li);
}
```

Programming in Modern C++



# numeric Component

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm  
copy

**numeric**  
accumulate  
inner\_product

functional

Module Summary

## numeric Component

### Source:

- [Numerics library](#), cplusplus
- `<numeric>`, cplusplus



# numeric Component

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm  
copy

numeric  
accumulate  
inner\_product

functional

Module Summary

- The header `<numeric>` defines a set of algorithms ( as function templates) to perform certain operations on sequences of numeric values
- Due to their flexibility, they can also be adapted for other kinds of sequences
- The component contains the following algorithms

`accumulate`

Accumulate values in range

`adjacent_difference`

Compute adjacent difference of range

`inner_product`

Compute cumulative inner product of range

`partial_sum`

Compute partial sums of range

`iota` [C++11]

Store increasing sequence



# accumulate

Module M45

Partha Pratim Das

Objectives & Outlines

Data Structures / Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

The `accumulate` is available in `<numeric>` and it

- Returns the result of accumulating all the values in the range `[first,last)` to `init`
- Uses `add` as default operation, but a different operation can be specified as `binary_op` (`BinOp`)

```
// default
```

```
template<class In, class T> T accumulate(In first, In last, T init) {  
    while (first!=last) {  
        init = init + *first; // init accumulates the result  
        ++first;  
    }  
    return init;  
}
```

```
// we do not need to use only +, we can use any binary operation (for example, *)
```

```
// any function that "updates the init value" can be used:
```

```
template<class In, class T, class BinOp> T accumulate(In first, In last, T init, BinOp op) {  
    while (first!=last) {  
        init = op(init, *first); // means "init op *first"  
        ++first;  
    }  
    return init;  
}
```



# accumulate: Sum the elements of a sequence

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate  
inner\_product

functional

Module Summary

```
#include <iostream>
#include <vector>
#include <numeric> // accumulate
using namespace std;

void f(vector<double>& vd, int* p, int n) {
    double sum = accumulate(vd.begin(), vd.end(), 0.0); // 12.3 : add the elements of vd
    // note: the type of the 3rd argument, the initializer, determines the precision used

    int si = accumulate(p, p+n, 0); // 10 : sum the ints in an int. p+n means (roughly) &p[n]

    long sl = accumulate(p, p+n, long(0)); // 10 : sum the ints in a long

    double s2 = accumulate(p, p+n, 0.0); // 10 : sum the ints in a double

    // popular idiom, use the variable you want the result in as the initializer:
    double ss = 0;
    ss = accumulate(vd.begin(), vd.end(), ss); // 12.3 : do remember the assignment
}

int main() { vector<int> v = { 1, 2, 3, 4 };
    int sum = accumulate(v.begin(), v.end(), 0); // 10

    vector<double> vd = { 1.5, 2.7, 3.2, 4.9 };
    f(vd, &v[0], v.size());
}
```

Programming in Modern C++





# accumulate: Multiply the elements of a sequence

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm  
copy

numeric  
accumulate  
inner\_product

functional

Module Summary

```
// often, we need multiplication rather than addition:
#include <iostream>
#include <list>
#include <numeric>    // accumulate
#include <functional> // multiplies
using namespace std;

void f(list<int>& ld) {
    int product = accumulate(ld.begin(), ld.end(),
                             1.0, // initializer 1.0
                             multiplies<int>()); // multiplies is an STL function object for multiplying

    cout << product << endl; // 24
}

int main() {
    list<int> l = { 1, 2, 3, 4 };

    f(l);
}
```



# accumulate: What if the data is part of a record?

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

```
struct Record {
    int units;                // number of units sold
    double unit_price;
    // ...
};

// let the "update the init value" function extract data from a Record element:
double price(double v, const Record& r) {
    return v + r.unit_price * r.units;
}

void f(const vector<Record>& vr) {
    double total = accumulate(vr.begin(), vr.end(), 0.0, price);
    // ...
}

void f(const vector<Record>& vr) {
    double total = accumulate(vr.begin(), vr.end(), 0.0, // use a lambda [C++11]
        [](double v, const Record& r) { return v + r.unit_price * r.units; }
    );
    // ...
}

// Is this clearer or less clear than the price() function?
```



# inner\_product

Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

The `inner_product` is available in `<numeric>` and it

- Computes cumulative inner product of range
- Returns the result of accumulating `init` with the inner products of the pairs formed by the elements of two ranges starting at `first1` and `first2`
- Uses two default operations (to add up the result of multiplying the pairs) that may be overridden by the arguments `binary_op1` (`BinOp`) and `binary_op2` (`BinOp2`)

```
template<class In, class In2, class T> T inner_product(In first, In last, In2 first2, T init) {
    // This is the way we multiply two vectors (yielding a scalar)
    while(first != last) {
        init = init + (*first) * (*first2);    // multiply pairs of elements and sum
        ++first; ++first2;
    }
    return init;
}

// we can supply our own operations for combining element values with "init":
template<class In, class In2, class T, class BinOp, class BinOp2 >
T inner_product(In first, In last, In2 first2, T init, BinOp op, BinOp2 op2) {
    while(first!=last) {
        init = op(init, op2(*first, *first2)); // In default op = operator+ and op2 = operator*
        ++first; ++first2;
    }
    return init;
}
```

Programming in Modern C++



# inner\_product

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

```
#include <iostream>
#include <vector>
#include <numeric> // inner_product
using namespace std;

int main() {
    // calculate the Dow-Jones industrial index:

    // share price for each company
    vector<double> dow_price = { 81.86, 34.69, 54.45 };

    // weight in index for each company
    vector<double> dow_weight = { 5.8549, 2.4808, 3.8940 };

    // multiply (price, weight) pairs and add
    double dj_index = inner_product(
        dow_price.begin(), dow_price.end(), dow_weight.begin(), 0.0);

    cout << dj_index << endl; // 777.369
}
```



# functional Component

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

**functional**

Module Summary

## functional Component

### Source:

- Standard library header `<functional>`, `cppreference`
- `<functional>`, `cplusplus`



# functional Component

## Module M45

Partha Pratim Das

Objectives & Outlines

Data Structures / Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

**functional**

Module Summary

- The header `<functional>` defines a set of useful function objects
- These are typically used as arguments to functions, such as *predicates* or *comparison functions* passed to standard algorithms
- Some useful standard function objects are:
  - Binary
    - ▷ `plus`, `minus`, `multiplies`, `divides`, `modulus`
    - ▷ `equal_to`, `not_equal_to`, `greater`, `less`, `greater_equal`, `less_equal`
    - ▷ `logical_and`, `logical_or`
  - Unary
    - ▷ `negate`, `logical_not`
- **C++11** has heavy use to function objects where the following will be discussed:
  - `function`, `bind`, `cref`, `ref`, `mem_fn`, ...



# Module Summary

## Module M45

Partha Pratim  
Das

Objectives &  
Outlines

Data Structures /  
Containers

Containers in C++

algorithm

copy

numeric

accumulate

inner\_product

functional

Module Summary

- Summarized containers in STL
- Glimpsed at [algorithm](#), [numeric](#), and [functional](#) library components



## Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

# Programming in Modern C++

Tutorial T09: How to design a UDT like built-in types?: Part 3: Updates and Mixes of UDTs

Partha Pratim Das

Department of Computer Science and Engineering  
Indian Institute of Technology, Kharagpur

*ppd@cse.iitkgp.ac.in*

*All url's in this module have been accessed in September, 2021 and found to be functional*





# Tutorial Recap

## Tutorial T09

Partha Pratim  
Das

## Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

- Presented the design, implementation and test for Int<N> and Poly<T> types
- Showed how Poly<int> as well as Poly<Fraction> works
- Outlined several practice UDTs for homework



# Tutorial Objectives

- To update UDTs: `Fraction`, `Int<N>` and `Poly<T>`
- To test mix of UDTs

Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

`Int<N>` UDT

Wraparound

Binary Ops

Mixed UDT Apps

`Fraction <int>`

`Fraction`  
`<Int<4> >`

`Poly<Int<4> >`

`Poly <Fraction`  
`<Int<N> > >`

Caveat

Tutorial Summary



# Tutorial Outline

## Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

- 1 Tutorial Recap
- 2 Update UDTs
- 3 Fraction UDT
  - friend Operators
  - Template
  - Mixed Format
- 4 Int<N> UDT
  - Wraparound
  - Binary Ops
- 5 Mixed UDT Apps
  - Fraction <int>
  - Fraction <Int<4> >
  - Poly<Int<4> >
  - Poly <Fraction <Int<N> > >
- 6 Caveat
- 7 Tutorial Summary



# Update UDTs

Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

## Update UDTs



# Update UDTs

Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template  
Mixed Format

Int<N> UDT

Wraparound  
Binary Ops

Mixed UDT Apps

Fraction <int>  
Fraction  
<Int<4> >  
Poly<Int<4> >  
Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

- **Fraction**

- Change binary arithmetic and comparison operators to **friend** functions from non-static member functions
- Parameterize **Fraction** with type **T = int**
- Provide mixed format support

- **Int<N>**

- In the constructor of **Int<N>**, allow out-of-range values to wrap around instead of **assert**
- Implement **operator\*()**, **operator/()**, and **operator%()**

- **Mixed UDT Apps**

- Test **Fraction<int>**
- Test **Fraction<Int<4> >**
- Test **Poly<Int<4> >**
- Test **Poly<Fraction<Int<4> > >**



# Fraction UDT: Update

Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

**Fraction UDT**

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

## Fraction UDT: Update



# Fraction UDT: Update: Agenda

## Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

**Fraction UDT**

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

- We have the following update agenda for **Fraction**
  - Change binary arithmetic and comparison operators to **friend** functions from non-static member functions
  - Parameterize **Fraction** with type **T = int**
  - Provide mixed format support



# Fraction: friend Operators

Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

- To facilitate the power of `friend` operators, we make the constructor non-`explicit`

```
/* explicit */ Fraction(int n = 1, int d = 1): // Three overloads
    n_(d < 0 ? -n : n), d_(d < 0 ? -d : d) // d_ cannot be -ve
{ *(*this); } // Reduces the fraction by operator*()
```

- Binary Arithmetic Operations: Add, Subtract, Multiply, Divide, and Modulus

```
friend Fraction operator+(const Fraction&, const Fraction&); // Add()
friend Fraction operator-(const Fraction&, const Fraction&); // Subtract()
friend Fraction operator*(const Fraction&, const Fraction&); // Multiply()
friend Fraction operator/(const Fraction&, const Fraction&); // Divide()
friend Fraction operator%(const Fraction&, const Fraction&); // Residue()
```

- Binary Relational Operations: Less, LessEq, More, MoreEq, Eq, NotEq

```
friend bool operator==(const Fraction& f1, const Fraction& f2); // Eq()
friend bool operator!=(const Fraction& f1, const Fraction& f2); // NotEq()
friend bool operator<(const Fraction& f1, const Fraction& f2); // Less()
friend bool operator<=(const Fraction& f1, const Fraction& f2); // LessEq()
friend bool operator>(const Fraction& f1, const Fraction& f2); // More()
friend bool operator>=(const Fraction& f1, const Fraction& f2); // MoreEq()
```





# Fraction: Template

## Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

- To provide an underlying type for `Fraction`, we introduce type variable `T` with `int` as default
- `T` could be any integral type like `int`, `short`, `char`, `long`, or `Int<N>` etc.
- We also change the name of the type from `Fraction` to `Fraction_` not to clutter the user name space

```
template<typename T = int>
class Fraction_ { public:

    // Change Fraction to Fraction_

    // ...
private:
    T      n_;          // The Numerator. Earlier: int
    T      d_;          // The Denominator. Earlier: unsigned int
    // ...
};
```

- In the application, add:

```
typedef Fraction_<int> Fraction; // Fraction is used in the application
```



# Fraction: Template

Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

```

template<typename T = int> class Fraction_ { public: // Parameterized Fraction_ with T = int
    Fraction_(T n = 1, T d = 1); Fraction_(const Fraction_& f);           // Ctor, C-Ctor, C=, Dtor
    ~Fraction_(); Fraction_& operator=(const Fraction_& f);
    friend ostream& operator<<(ostream& os, const Fraction_& f);           // Streaming ops
    friend istream& operator>>(istream& is, Fraction_& f);
    Fraction_ operator-() const; Fraction_ operator+() const;           // Unary arithmetic ops
    Fraction_ operator++(); Fraction_ operator++(int);
    Fraction_ operator--(); Fraction_ operator--(int);
    friend Fraction_ operator+(const Fraction_& f1, const Fraction_& f2); // Binary arithmetic ops
    friend Fraction_ operator-(const Fraction_& f1, const Fraction_& f2);
    friend Fraction_ operator*(const Fraction_& f1, const Fraction_& f2);
    friend Fraction_ operator/(const Fraction_& f1, const Fraction_& f2);
    friend Fraction_ operator%(const Fraction_& f1, const Fraction_& f2);
    friend bool operator==(const Fraction_& f1, const Fraction_& f2);     // Comparison ops
    friend bool operator!=(const Fraction_& f1, const Fraction_& f2);
    friend bool operator<(const Fraction_& f1, const Fraction_& f2);
    friend bool operator<=(const Fraction_& f1, const Fraction_& f2);
    friend bool operator>(const Fraction_& f1, const Fraction_& f2);
    friend bool operator>=(const Fraction_& f1, const Fraction_& f2);
    Fraction_& operator+=(const Fraction_& f);                           // Advanced assignment ops
    Fraction_& operator-=(const Fraction_& f); Fraction_& operator*=(const Fraction_& f);
    Fraction_& operator/=(const Fraction_& f); Fraction_& operator%=(const Fraction_& f);
    Fraction_ operator!() const; operator double() const;               // Special ops
private: static T gcd(T a, T b); static T lcm(T a, T b); Fraction_& operator*(); // Support functions
};

```

Programming in Modern C++



# Fraction: Mixed Format Support

## Tutorial T09

Partha Pratim Das

## Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

- Irrespective of whether a fraction is in *simple format* like  $\frac{n}{d}$  ( $\frac{2}{3}$  or  $\frac{17}{5}$ ) or in *mixed format* like  $w\frac{n}{d}$  ( $\frac{2}{3}$  or  $3\frac{2}{5}$ ), its *internal representation is always simple*  $w\frac{n}{d} \equiv \frac{w*d+n}{d}$
- Hence, mixed format support is limited to:
  - Fraction construction

```
explicit Fraction_(T w, T n, T d) : // Mixed format fraction constructor
    n_(d < 0 ? w * -d - n : w * d + n), d_(d < 0 ? -d : d) // d must be non-negative
{
    (*this);
} // Reduces the fraction
```
  - Fraction output operator

```
friend ostream& operator<<(ostream& os, const Fraction_& f);
```
  - Fraction input operator

```
friend istream& operator>>(istream& is, Fraction_& f);
```
- While the constructor can be distinguished by the distinct signature, the streaming operators have the same signature for *simple* as well as *mixed format*. Hence, we need a way to tell these operators about the format



# Fraction: Mixed Format Support in Streaming Operators

## Tutorial T09

Partha Pratim Das

## Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

- To design for the mixed format i/o, we recall the support for writing integers in multiple bases using `<iomanip>` component in standard library

```
#include <iostream>
#include <iomanip>
int main() { int i = 76;
    std::cout << std::oct << i << std::endl; // Set octal format. Prints 114
    std::cout << std::hex << i << std::endl; // Set hexadecimal format. Prints 4c
    std::cout << std::dec << i << std::endl; // Set decimal format. Prints 76
}
```

- Using `<iomanip>`, the format flag is set in `ostream` (`cout`). We cannot do that as `ostream` (or `istream`) cannot be changed. So, we need to keep the format option in the `Fraction_` class
- We add a `static bool bMixedFormat_` (`true` for mixed format, `false` for simple format)
- In the streaming operators, we can check this flag and adopt the appropriate formatting
- But how do we set / reset this flag? Using `SetFormat(bool)` spoils the built-in type-like syntax

Easy

Desired

```
Fraction f(17,5);
Fraction::SetFormat(false);
cout << f; // 17/5
Fraction::SetFormat(true);
cout << f; // 3+2/5
```

```
Fraction f(17,5);
cout << Fraction::simple;
cout << f; // 17/5
cout << Fraction::mixed;
cout << f; // 3+2/5
```



# Fraction: Mixed Format Support in Streaming Operators

Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction <Int<4> >

Poly<Int<4> >

Poly <Fraction <Int<N> > >

Caveat

Tutorial Summary

- For `cout << Fraction::simple` (`cout << Fraction::mixed`), we observe the following:
  - `Fraction::simple` (`Fraction::mixed`) needs to have an appropriate type, say `Format`, different from `Fraction_`, yet encapsulated by `Fraction_`. So we choose nested `class Format` in `Fraction_`

```
class Format { bool bFormat_; public: Format(bool b): bFormat_(b) { } /* ... */ }; // Wraps bool
```
  - `Fraction::simple` and `Fraction::mixed` must be constants in `Fraction_`

```
static const Format mixed; // bMixedFormat_ = true
static const Format simple; // bMixedFormat_ = false
```
  - Output streaming `Fraction::simple` (`Fraction::mixed`) in `Fraction::Format` should print nothing and set `Fraction::bMixedFormat_` appropriately
 

```
friend ostream& operator<<(ostream& os, const Format& m) { // writes nothing
    // sets / resets mixed format flag
    bMixedFormat_ = m.bFormat_; // error: operator<< is friend of Format, not of Fraction_
    return os;
}
```
  - So we use a wrapper in `Format`

```
class Format { // ...
    void SetMixedFormat(bool b) const { bMixedFormat_ = b; } // access private member of Fraction_
    friend ostream& operator<<(ostream& os, const Format& m) { // writes nothing
        m.SetMixedFormat(m.bFormat_); // sets / resets mixed format flag
        return os;
    }
};
```



# Fraction: Mixed Format Support in Streaming Operators

Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

- We use `Fraction_::bMixedFormat_` to decide the format in the streaming operators:

```
friend ostream& operator<<(ostream& os, const Fraction_& f) {
    T w = 0, n = f.n_, d = f.d_;
    if (f.bMixedFormat_) { // Mixed format support
        w = n / d;          // Whole part = 3 in 17/5
        n %= d;             // Fraction part = 2/5 in 17/5
        if (n < 0) { --w; n += d; } // Negative: -17/5 = -4+3/5 = (-17/5 -1)+(-17%5+5)/5
        if (w) os << w << "+";    // w+ to be suppressed if 0
    }
    os << n; if ((n != 0) && (d != 1)) os << "/" << d; // To print the fraction part in both formats
    return os;
}

friend istream& operator>>(istream& is, Fraction_& f) {
    if (f.bMixedFormat_) { // Mixed format support - reads 3 numbers: w, n, d
        cout << "Input fraction in mixed Format" << endl;
        T w, n;
        is >> w >> n >> f.d_;
        f.n_ = w * f.d_ + n;
    }
    else // Simple format support - reads 2 numbers: n, d
        is >> f.n_ >> f.d_;
    *f; // Reduces the fraction
    return is;
}
```



# Fraction: Mixed Format Support in Streaming Operators

## Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

- Finally, we put together `Fraction_::Format` class:

```
template<typename T = int> class Fraction_ { public: // ...
private: /* ... */ // Support for Mixed Format
    class Format { private: // Wraps bool so that special IO operators can be defined
        bool bFormat_; // Truthvalue for Format object
        void SetMixedFormat(bool b) const; // Sets Fraction_::bMixedFormat_
        Format(bool b): bFormat_(b) { } // Ctor is private - used only by friend class Fraction_

        friend ostream& operator<<(ostream& os, const Format& m); // Called to set bMixedFormat_
        friend istream& operator>>(istream& is, const Format& m); // Called to set bMixedFormat_

        friend class Fraction_; // Since ctor of Format is private, Fraction_ must be a friend
    };
public:
    // Format markers
    static const Format mixed; // Denotes bMixedFormat_ = true
    static const Format simple; // Denotes bMixedFormat_ = false
    // ...
};
```

- Instantiations of Format markers are:

```
const Fraction::Format Fraction::mixed(true); // Denotes bMixedFormat_ = true
const Fraction::Format Fraction::simple(false); // Denotes bMixedFormat_ = false
```



# Int<N> UDT: Update

Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

**Int<N> UDT**

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

## Int<N> **UDT**: Update





# Int<N> UDT: Update: Agenda

## Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

**Int<N> UDT**

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

- We have the following update agenda for `Int<N>`
  - In the constructor of `Int<N>`, allow out-of-range values to wrap around instead of `assert`
  - Implement `operator*()`, `operator/()`, and `operator%()`



# Int<N>: Constructor with Wraparound

## Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction <int<4> >

Poly<int<4> >

Poly <Fraction <int<N> > >

Caveat

Tutorial Summary

- The constructor of `Int<N>` is:

```
template<typename T = int, unsigned int N = 4>
class Int_ { public: // ...
    explicit Int_<T, N>(int v = 1): v_(v) { // Two overloads of Constructor
        assert(v_ <= static_cast<int>(MaxInt)); // assert will fire if the value
        assert(v_ >= static_cast<int>(MinInt)); // is out of limits
    } // ...
};
```

- For wraparound, we remove `asserts` and overload `operator*()`

```
template<typename T = int, unsigned int N = 4>
class Int_ { public: // ...
    explicit Int_<T, N>(int v = 1): v_(v) // Two overloads of Constructor
    { *(*this); }
    Int_<T, N> operator*() { // Wraparound operator
        v_ = v_ % TwoPowerN_T;
        if (v_ > MaxInt_T) v_ -= TwoPowerN_T;
        else if (v_ < MinInt_T) v_ += TwoPowerN_T;
        return *this;
    } // ...
};
```

- With this we get the following wraparound:

```
cout << Int_<>(5) << ' ' << Int_<>(77) << ' ' << Int_<>(-43) << endl; // 5 -3 5
```



# Int<N>: Binary Operators

Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

- With the wraparound, it becomes straightforward to implement binary operators with overflow

```
template<typename T = int, unsigned int N = 4>class Int_ { public: // ...
    friend Int_<T, N> operator+(const Int_<T, N>& i1, const Int_<T, N>& i2) {
        return Int_<T, N>(i1.v_ + i2.v_); }
    friend Int_<T, N> operator-(const Int_<T, N>& i1, const Int_<T, N>& i2) {
        return i1 + (-i2); } // return Int_<T, N>(i1.v_ - i2.v_); is also okay
    friend Int_<T, N> operator*(const Int_<T, N>& i1, const Int_<T, N>& i2) {
        return Int_<T, N>(i1.v_ * i2.v_); }
    friend Int_<T, N> operator/(const Int_<T, N>& i1, const Int_<T, N>& i2) {
        return Int_<T, N>(i1.v_ / i2.v_); }
    friend Int_<T, N> operator%(const Int_<T, N>& i1, const Int_<T, N>& i2) {
        return Int_<T, N>(i1.v_ % i2.v_); } // ...
};
```

- With this we get the following:

```
cout << "Binary Plus: Int(2) + Int(3) = " << (Int(2) + Int(3)) << endl; // 5
cout << "Binary Plus: Int(-6) + Int(-7) = " << (Int(-6) + Int(-7)) << endl; // 3
cout << "Binary Minus: Int(2) - Int(3) = " << (Int(2) - Int(3)) << endl; // -1
cout << "Binary Minus: Int(-6) - Int(-7) = " << (Int(-6) - Int(-7)) << endl; // 1
cout << "Binary Multiply: Int(3) * Int(2) = " << (Int(3) * Int(2)) << endl; // 6
cout << "Binary Multiply: Int(7) * Int(5) = " << (Int(7) * Int(5)) << endl; // 3
cout << "Binary Multiply: Int(-8) * Int(-8) = " << (Int(-8) * Int(-8)) << endl; // 0
cout << "Binary Divide: Int(3) / Int(2) = " << (Int(3) / Int(2)) << endl; // 1
cout << "Binary Divide: Int(7) / Int(-5) = " << (Int(7) / Int(-5)) << endl; // -1
cout << "Binary Residue: Int(3) % Int(2) = " << (Int(3) % Int(2)) << endl; // 1
cout << "Binary Residue: Int(-6) % Int(2) = " << (Int(-6) % Int(2)) << endl; // 0
```



# Int<N>: Binary Operators: Properties

## Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

- Try to prove the usual arithmetic properties of the `Int<N>` binary operators for addition, subtraction, multiplication, and division under wraparound:
  - Are all operators *Associative*? For example,  
$$\triangleright a + b + c = (a + b) + c = a + (b + c)$$
  - Are addition and multiplication *Commutative*? For example,  
$$\triangleright a + b = b + a$$
  - Do multiplication and division *Distribute* over addition and subtraction? For example,  
$$\triangleright a * (b + c) = a * b + a * c$$
- Especially, check for the boundary conditions under wraparound:
  - `MaxInt` + 1 = `MinInt`
  - `MinInt` - 1 = `MaxInt`
  - - `MinInt` = `MinInt`
- Consider exception and / or assert support in the constructors and / or operators if some specific values can break the properties



# Mixed UDT Apps

Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

## Mixed UDT Apps



# Mixed UDT Apps: Agenda

## Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

- We have the following agenda for Mixed UDT Apps
  - Test `Fraction<int>`
  - Test `Fraction<Int<4> >`
  - Test `Poly<int>`: Done in **Tutorial 08**
  - Test `Poly<Fraction<int > >`: Done in **Tutorial 08** - actually using the non-template version of `Fraction`
  - Test `Poly<Int<4> >`
  - Test `Poly<Fraction<Int<4> > >`



# Fraction<int>: Application

Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

```
#include <iostream>
using namespace std;
#include "Frac.h"

typedef Fraction<int> Fraction;
const Fraction Fraction::UNITY = Fraction(1), Fraction::ZERO = Fraction(0);
bool Fraction::bMixedFormat_ = false;
const Fraction::Format Fraction::mixed(true), Fraction::simple(false);

int main() {
    Fraction fa(5, 3);
    cout << "Fraction fa(5, 3) = " << Fraction::mixed << fa << " = " << Fraction::simple << fa;
    Fraction fb(7, 9);
    cout << "Fraction fb(7, 9) = " << Fraction::mixed << fb << " = " << Fraction::simple << fb;
    cout << "fa + fb = " << Fraction::mixed << (fa + fb) << " = " << Fraction::simple << (fa + fb);
    cout << "fa - fb = " << Fraction::mixed << (fa - fb) << " = " << Fraction::simple << (fa - fb);
    cout << "fa * fb = " << Fraction::mixed << (fa * fb) << " = " << Fraction::simple << (fa * fb);
    cout << "fa / fb = " << Fraction::mixed << (fa / fb) << " = " << Fraction::simple << (fa / fb);
}
```

Fraction fa(5, 3) =  $1+2/3 = 5/3$

Fraction fb(7, 9) =  $7/9 = 7/9$

fa + fb =  $2+4/9 = 22/9$

fa - fb =  $8/9 = 8/9$

fa \* fb =  $1+8/27 = 35/27$

fa / fb =  $2+1/7 = 15/7$

Programming in Modern C++



# Fraction<Int<4> >: Application

Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction

<Int<4> >

Poly<Int<4> >

Poly <Fraction

<Int<N> > >

Caveat

Tutorial Summary

```

#include <iostream>
using namespace std;
#include "Frac.h"
#include "../Int/Int.h"
typedef Int<int, 4> Int; typedef Fraction<Int> Fraction;
const Fraction Fraction::UNITY = Fraction(1), Fraction::ZERO = Fraction(0);
bool Fraction::bMixedFormat_ = false;
const Fraction::Format Fraction::mixed(true), Fraction::simple(false);

int main() {
    Fraction fa(5, 3);
    cout << "Fraction fa(5, 3) = " << Fraction::mixed << fa << " = " << Fraction::simple << fa;
    Fraction fb(7, 10);
    cout << "Fraction fb(7, 10) = " << Fraction::mixed << fb << " = " << Fraction::simple << fb;
    cout << "fa + fb = " << Fraction::mixed << (fa + fb) << " = " << Fraction::simple << (fa + fb);
    cout << "fa - fb = " << Fraction::mixed << (fa - fb) << " = " << Fraction::simple << (fa - fb);
    cout << "fa * fb = " << Fraction::mixed << (fa * fb) << " = " << Fraction::simple << (fa * fb);
    cout << "fa / fb = " << Fraction::mixed << (fa / fb) << " = " << Fraction::simple << (fa / fb);
}

```

Fraction fa(5, 3) =  $1+2/3 = 5/3$

Fraction fb(7, 10) =  $-2+5/6 = -7/6$

fa + fb =  $1/2 = 1/2$

fa - fb =  $1/6 = 1/6$

fa \* fb =  $-2+1/2 = -3/2$

fa / fb =  $2/5 = 2/5$

Programming in Modern C++





# Poly<Int<4> >: Application

Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

```
#include <iostream>
using namespace std;
#include "../Int/Int.h"
#include "Poly.h"
typedef Int<int, 4> Int;
const int Int::TwoPowerN_T = 1 << N;           // 2^N
const int Int::MaxInt_T = (1 << (N-1))-1; /* 2^(N-1)-1 */ Int::MinInt_T = -(1 << (N-1)); // -2^(N-1)
const Int Int::MaxInt = Int(Int::pow() - 1), Int::MinInt = Int(-Int::pow());

void main() { vector<Int> vf = { 2, 15, 7 };
    Poly<Int> pf1(vf); cout << "pf1(x): " << pf1 << " pf1(2) = " << pf1(2) << endl;
    Poly<Int> pf2; cout << "pf2(x): " << pf2 << " pf2(2) = " << pf2(2) << endl;
    cin >> pf2; /* 3 9 7 2 -11 */ cout << "pf2(x): " << pf2 << " pf2(2) = " << pf2(2) << endl;
    Poly<Int> pf3 = pf1 + pf2; cout << "pf3(x): " << pf3 << " pf3(2) = " << pf3(2) << endl;
    Poly<Int> pf4 = pf1 - pf2; cout << "pf4(x): " << pf4 << " pf4(2) = " << pf4(2) << endl << endl;
}
```

pf1(x):  $7x^2 + -1x^1 + 2$ . pf1(2) = -4 // 2 = 2, 15 = -1, 7 = 7. pf1(2) =  $7*4 + -1*2 + 2 = 28-2+2 = 28 = -4$   
 pf2(x): 1. pf2(2) = 1  
 Enter degree of the polynomial 3  
 Enter all the coefficients like  $a_0+a_1x+a_2x^2+...a_nx^n$   
 9 7 2 -11 // -7 7 2 5  
 pf2(x):  $5x^3 + 2x^2 + 7x^1 + -7$ . pf2(2) = 7 // pf2(2) =  $5*8 + 2*4 + 7*2 + -7 = 56+8+14-7 = 71 = 71-64 = 7$   
 pf3(x):  $5x^3 + -7x^2 + 6x^1 + -5$ . pf3(2) = 3  
 pf4(x):  $-5x^3 + 5x^2 + -8x^1 + -7$ . pf4(2) = 5



# Poly<Fraction<Int<4> > >: Application

Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

```
#include <iostream>
using namespace std;
#include "../Fraction/Frac.h"
#include "../Int/Int.h"
#include "Poly.h"
const int N = 4; typedef Int_<int, 4> Int; const int Int::TwoPowerN_T = 1 << N; // 2^N
const int Int::MaxInt_T = (1 << (N-1))-1; /* 2^(N-1)-1 */ Int::MinInt_T = -(1 << (N-1)); // -2^(N-1)
const Int Int::MaxInt = Int(Int::pow() - 1), Int::MinInt = Int(-Int::pow());
typedef Fraction_<Int> Fraction; bool Fraction::bMixedFormat_ = false;
const Fraction Fraction::UNITY = Fraction(1), Fraction::ZERO = Fraction(0);
const Fraction::Format Fraction::mixed(true), Fraction::simple(false);
void main() {
    vector<Fraction> vf1 = { Fraction(1, 2), Fraction(-3, 5), Fraction(2, 4) };
    Poly<Fraction> pf1(vf1); cout << "pf1(x): " << pf1 << " pf1(Fraction(2)) = " << pf1(Fraction(2)) << endl;
    vector<Fraction> vf2 = { Fraction(1, 2), Fraction(2, 3) };
    Poly<Fraction> pf2(vf2); cout << "pf2(x): " << pf2 << " pf2(Fraction(2)) = " << pf2(Fraction(2)) << endl;
    Poly<Fraction> pf3 = pf1 + pf2;
    cout << "pf3(x): " << pf3 << " pf3(Fraction(2)) = " << pf3(Fraction(2)) << endl;
    Poly<Fraction> pf4 = pf1 - pf2;
    cout << "pf4(x): " << pf4 << " pf4(Fraction(2)) = " << pf4(Fraction(2)) << endl << endl;
}
pf1(x): 1/2x^2 + -3/5x^1 + 1/2. pf1(Fraction(2)) = 7/6 // pf1(2/1) = 1/2*4 -3/5*2 + 1/2 = 7/6
pf2(x): 2/3x^1 + 1/2. pf2(Fraction(2)) = -5/6 // pf2(2/1) = 2/3*2/1 + 1/2 = 4/3 + 1/2 = 11/6 = -5/6
pf3(x): 1/2x^2 + 1. pf3(Fraction(2)) = 3 // pf4(2/1) = 1/2*4/1 + 1 = 3
pf4(x): 1/2x^2. pf4(Fraction(2)) = 2 // pf4(2/1) = 1/2*4/1 = 2
```



# Caveat: Mixes may fail

Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

## Caveat: Mixes may fail



# Caveat in mixing UDTs

## Tutorial T09

Partha Pratim Das

Tutorial Recap

Objective & Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction <Int<4> >

Poly<Int<4> >

Poly <Fraction <Int<N> > >

Caveat

Tutorial Summary

- While `Fraction<int>`, `Poly<int>`, `Int<N>`, or `Poly<Fraction<int> >` work perfectly fine, `Fraction<Int<N> >` or `Poly<Fraction<Int<N> > >` may have some surprise
- This is due to the `T gcd(T, T)` algorithm in the context of `Int<N>`. Normally, we invoke `gcd()` for positive numbers only (that's how the Euler's Algorithm is designed to work)
- However, for `MinInt` in `Int<N>`, we have `-MinInt = MinInt`. Hence, if one of the `gcd()` parameters is `MinInt` we are perpetually in the realm of negative numbers. This leads to an *infinite loop* in the code below:

```
static T gcd(T a, T b) { // Finds the gcd for two +ve integers
    while (a != b) if (a > b) a = a - b; else b = b - a; // N = 4. (-8,3) => (-8,-5) => (-8,3) => ...
    return a;
}
```

- So we choose to `throw` (and eventually `assert` in the constructor) when one of the `gcd()` arguments is negative (eventually `MinInt`)

```
static T gcd(T a, T b) { // Finds the gcd for two +ve integers
    if (a < 0) throw "Negative first arg in gcd";
    if (b < 0) throw "Negative second arg in gcd";
    while (a != b) if (a > b) a = a - b; else b = b - a; // For N = 4, a = -8 is an infinite loop
    return a;
}
```

- How to fix?



# Tutorial Summary

## Tutorial T09

Partha Pratim  
Das

Tutorial Recap

Objective &  
Outline

Update UDTs

Fraction UDT

friend Operators

Template

Mixed Format

Int<N> UDT

Wraparound

Binary Ops

Mixed UDT Apps

Fraction <int>

Fraction  
<Int<4> >

Poly<Int<4> >

Poly <Fraction  
<Int<N> > >

Caveat

Tutorial Summary

- UDTs Fraction, Int and Poly have been updated with various features
- Mixed applications involving multiple UDTs have been checked
- Caveat or loophole has been identified